МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики
(повне найменування назва факультету)

Кафедра інформаційних систем
(повна назва кафедри)

КВАЛІФІКАЦІЙНА (БАКАЛАВРСЬКА) РОБОТА

РОЗРОБКА ІНТЕРАКТИВНОЇ КАРТОГРАФІЧНОЇ СИСТЕМИ ДЛЯ НАВІГАЦІЇ УНІВЕРСИТЕТОМ

Виконав(ла): студент(ка) групи ПМІ-42
спеціальності 122 – комп'ютерні науки
(шифр і назва спеціальності)

Зозуля О. Ю.
(підпис) (прізвище та ініціали)

Керівник Козій І. Я.
(підпис) (прізвище та ініціали)

Рецензент
(підпис) (прізвище та ініціали

2026

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет *прикладної математики та інформатики*

Кафедра *інформаційних систем*

Освітньо-кваліфікаційний рівень *бакалавр*

Напрям підготовки

Спеціальність *122 Комп'ютерні науки*
(шифр і назва)

«ЗАТВЕРДЖУЮ»

В.о. завідувача кафедри

доц. В.М. Горлач

"8" вересня 2025 року

З А В Д А Н Н Я

НА КВАЛІФІКАЦІЙНУ (БАКАЛАВРСЬКУ) РОБОТУ СТУДЕНТА

Зозулі Олега Юрійовича
(прізвище, ім'я, по батькові)

1. Тема роботи

Розробка інтерактивної картографічної системи для навігації університетом

керівник роботи *доц. Козій І. Я.*,
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені Вченою радою факультету від *"6" лютого 2026 року №24*

2. Строк подання студентом роботи *11.06.2026*

3. Вихідні дані до роботи *наукові джерела з теорії графів та алгоритмів пошуку найкоротшого шляху, офіційна документація використаних технологій (Node.js, NestJS, PostgreSQL, React, Next.js)*

4. Зміст кваліфікаційної (бакалаврської) роботи (перелік питань, які потрібно розробити)

1) *проаналізувати наявні рішення для навігації приміщеннями та сформулювати вимоги до системи;*
2) *обґрунтувати вибір набору технологій та методів реалізації, зокрема алгоритму пошуку найкоротшого шляху;*
3) *спроектувати архітектуру системи та модель даних, зокрема графову модель плану поверху;*
4) *реалізувати серверну та клієнтську частини застосунку;*
5) *виконати тестування та перевірку працездатності розробленого продукту.*

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)
загальна архітектура системи, діаграма «сутність — зв’язок» бази даних, діаграма варіантів використання, діаграми послідовностей основних процесів, зображення інтерфейсу застосунку.

6. Консультанти розділів роботи

Розділ	**Прізвище, ініціали та посада консультанта**	**Підпис, дата**	
		завдання видав	**завдання прийняв**

7. Дата видачі завдання *15.09.25*

КАЛЕНДАРНИЙ ПЛАН

№ з/п	**Назва етапів кваліфікаційної (бакалаврської) роботи**	**Строк виконання етапів роботи**	**Примітка**
1.	*Дослідження предметної області та наявних рішень, формулювання вимог*	*вересень 2025*	*виконано*
2.	*Вибір та аналіз технологій і методів*	*жовтень 2025*	*виконано*
3.	*Проєктування архітектури системи та моделі даних*	*листопад 2025*	*виконано*
4.	*Розробка бекенд частини застосунку*	*грудень 2025 - січень 2026*	*виконано*
5.	*Розробка фронтенд частини застосунку*	*лютий - березень 2026*	*виконано*
6.	*Тестування застосунку та оптимізація*	*квітень 2026*	*виконано*
7.	*Оформлення бакалаврської роботи*	*травень – червень 2026*	*виконано*

Студент _____ _____
(підпис) (прізвище та ініціали)

Керівник роботи _____
(підпис) (прізвище та ініціали)

ЗМІСТ

ВСТУП 7
РОЗДІЛ 1 ПОСТАНОВКА ЗАДАЧІ 10
1.1 Предметна область та її особливості 10
1.2 Аналіз наявних рішень 11
1.2.1 Статичні плани та інформаційні стенди 12
1.2.2 Google Maps та загальні картографічні сервіси 12
1.2.3 Спеціалізовані платформи (MazeMap та аналоги) 12
1.3 Функціональні вимоги 13
1.4 Нефункціональні вимоги 14
1.5 Ролі користувачів 14
1.6 Формулювання задачі 15
1.7 Припущення та обмеження 15
РОЗДІЛ 2 ВИКОРИСТАНИЙ НАБІР ТЕХНОЛОГІЙ ТА МЕТОДІВ 17
2.1 TypeScript 17
2.2 Node.js 17
2.3 NestJS та Fastify 18
2.4 PostgreSQL та Drizzle ORM 18
2.5 GraphQL (Mercurius) та REST 19
2.6 Автентифікація: JWT та bcrypt 19
2.7 Redis та BullMQ 20
2.8 Об’єктне сховище (S3-сумісне) 20
2.9 Next.js та React 20
2.10 Tailwind CSS та Material Design 3 20

2.11 Керування станом: React Query та Zustand 21
2.12 Допоміжні бібліотеки 21
2.13 Засоби тестування 22
2.14 Інструменти забезпечення якості та збирання 22
2.15 Алгоритм для пошуку найкоротшого шляху на графі A* 22
РОЗДІЛ 3 ПРОЕКТУВАННЯ СИСТЕМИ ТА МОДЕЛЬ ДАНИХ 24
3.1 Загальна архітектура системи 24
3.2 Модель даних 25
3.3 Модель плану поверху як графа 27
3.4 Перевірка цілісності та зв’язності карт 29
3.5 Варіанти використання 29
3.6 Діаграми послідовностей основних процесів 30
3.7 Версіонування карт та узгодженість даних 32
3.8 Безпека та розмежування доступу на рівні проектування 33
РОЗДІЛ 4 ОСОБЛИВОСТІ ПРОГРАМНОЇ РЕАЛІЗАЦІЇ 34
4.1 Структура серверної частини 34
4.2 Прикладний інтерфейс: REST та GraphQL 35
4.3 Автентифікація та авторизація 36
4.4 Підтвердження електронної пошти 37
4.5 Завантаження та валідація карт 37
4.6 Побудова найкоротшого шляху 38
4.7 Пошук приміщень і категорії 39
4.8 Історія пошуків і маршрутів 40
4.9 Збережені маршрути та приміщення 40
4.10 Події університету 40

4.11 Адміністрування 41
4.12 Клієнтська частина 41
4.13 Тестування та перевірка працездатності 42
4.14 Обробка помилок, журналювання та конфігурація 45
4.15 Продуктивність та кешування 45
РОЗДІЛ 5 ОПИС ІНТЕРФЕЙСУ КОРИСТУВАЧА 46
5.1 Головна сторінка та інтерактивна карта 46
5.2 Пошук приміщень 47
5.3 Фільтрування за категоріями 48
5.4 Деталі приміщення 49
5.5 Навігація та побудова маршруту 50
5.6 Історія 52
5.7 Збережені приміщення та маршрути 54
5.8 Події університету 55
5.9 Вхід та реєстрація 56
5.10 Інтерфейс адміністратора 57
5.11 Перемикання мови, теми та поверху 60
5.12 Адаптивність та мобільний вигляд 61
ВИСНОВКИ 62
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 64

ВСТУП

Сучасні заклади вищої освіти — це складні просторові комплекси з десятками аудиторій, лабораторій, адміністративних та сервісних приміщень, розташованих на кількох поверхах одного або кількох корпусів. Для абітурієнтів, першокурсників, студентів заочної форми навчання та сторонніх відвідувачів орієнтування у такому середовищі часто перетворюється на окрему проблему: паперові плани при входах є статичними, не інтерактивними та швидко застарівають, а традиційні картографічні сервіси, такі як Google Maps, орієнтовані передусім на відкритий простір і не деталізують внутрішню структуру будівель.

Останніми роками задача навігації всередині приміщень (англ. indoor navigation) виокремилася в самостійний напрям. На відміну від супутникової навігації на відкритій місцевості, у приміщеннях сигнал глобальних навігаційних систем є ненадійним, тому основою рішення стає не геопозиціонування, а коректна цифрова модель простору — план поверху, поданий як граф прохідних шляхів. Якість такої моделі прямо визначає коректність побудованого маршруту, тому до неї висуваються суворі вимоги цілісності та зв’язності.

Веб-технології є природним середовищем для подібного застосунку: він доступний із будь-якого пристрою без встановлення, легко оновлюється централізовано та підтримує адаптивний інтерфейс для настільних і мобільних екранів. Поєднання сучасного серверного каркаса на основі Node.js [21], реляційної бази даних PostgreSQL [22] та клієнтського застосунку на React [23] дає змогу побудувати продуктивне, безпечне й розширюване рішення, що відповідає інженерним стандартам галузі [7].

Усе викладене зумовлює актуальність роботи: розроблення доступного веб-застосунку інтерактивної навігації приміщеннями університету, який поєднує точну графову модель поверхів, пошук аудиторій, прокладання найкоротшого маршруту між приміщеннями та зручний, схожий на Google Maps інтерфейс користувача.

Об’єктом дослідження є процес орієнтування користувача у багатоповерховій будівлі університету та інформаційне забезпечення цього процесу. Предметом дослідження є моделі, методи та програмні засоби побудови цифрової навігаційної системи приміщень: модель плану поверху у вигляді зв’язного графа, алгоритми пошуку найкоротшого шляху та архітектурні рішення для веб-застосунку, що надає ці можливості кінцевому користувачеві.

Практичне значення роботи полягає у створенні готового до використання веб-застосунку, який скорочує час пошуку приміщень, знижує навантаження на довідкові служби закладу та покращує враження відвідувачів. Завдяки повному контролю закладу над картографічними даними та відкритій до розширення архітектурі систему можна поступово поширювати на нові корпуси без залучення сторонніх постачальників і без істотних змін у коді.

Метою кваліфікаційної роботи є проектування та розробка веб-застосунку для інтерактивної навігації приміщеннями університету, що забезпечує пошук аудиторій і побудову найкоротших маршрутів на основі зв’язної графової моделі поверхів та надає сучасний адаптивний інтерфейс користувача.

Визначена мета зумовила постановку та розв’язання таких завдань:

- проаналізувати наявні рішення для навігації приміщеннями та сформулювати вимоги до системи;
- обґрунтувати вибір набору технологій для серверної та клієнтської частин застосунку;
- спроектувати архітектуру системи та модель даних, зокрема модель плану поверху як зв’язного неорієнтованого графа;
- розробити серверну частину: автентифікацію та авторизацію, завантаження й валідацію карт, пошук кімнат, побудову маршрутів та роботу з історією;
- розробити клієнтську частину з інтерактивною картою, пошуком, фільтрами категорій, навігацією та адміністративним інтерфейсом;
- виконати тестування розробленого продукту та підтвердити його працездатність.

У роботі застосовано методи об’єктно-орієнтованого та доменно-орієнтованого проектування [4], теорію графів і алгоритми пошуку найкоротшого шляху, зокрема алгоритм A* [6], [12], методи проектування реляційних баз даних [22], а також принципи проектування інтерфейсів користувача та забезпечення зручності використання [8], [10].

У першому розділі проаналізовано предметну область і наявні рішення, сформульовано функціональні та нефункціональні вимоги, визначено ролі користувачів і подано точну постановку задачі.

У другому розділі обґрунтовано вибраний набір технологій для серверної та клієнтської частин застосунку.

У третьому розділі описано архітектуру системи, модель даних та модель плану поверху як зв’язного неорієнтованого графа, наведено діаграми варіантів використання та послідовностей.

У четвертому розділі викладено особливості програмної реалізації серверної та клієнтської частин, а також тестування продукту.

У п’ятому розділі подано опис інтерфейсу користувача з ілюстраціями основних сценаріїв роботи із застосунком.

РОЗДІЛ 1
ПОСТАНОВКА ЗАДАЧІ

1.1 Предметна область та її особливості

Предметною областю роботи є навігація приміщеннями (indoor navigation) у межах університетського комплексу. Об’єктом дослідження є процес орієнтування користувача у багатоповерховій будівлі, а предметом — програмні методи й засоби, що забезпечують пошук приміщень і прокладання маршрутів між ними на основі цифрової моделі плану.

Університетський корпус характеризується великою кількістю однотипних приміщень, складною топологією коридорів, наявністю вертикальних переходів (сходів), що з’єднують поверхи, а також постійними змінами призначення аудиторій. Через це орієнтування ускладнюється навіть для людей, які регулярно перебувають у будівлі, не кажучи вже про абітурієнтів і гостей.

Ключова відмінність навігації у приміщеннях від навігації на відкритій місцевості полягає у відсутності надійного сигналу глобальних супутникових систем позиціонування всередині будівель. Тому основою розв’язання задачі стає не визначення координат користувача, а точна топологічна модель простору: план кожного поверху подають як граф, вершинами якого є характерні точки (входи до приміщень, перехрестя коридорів, сходові майданчики), а ребрами — прохідні відрізки шляху. Така модель є класичним об’єктом теорії графів, а пошук маршруту зводиться до задачі про найкоротший шлях у зваженому графі [3].

Важливою властивістю моделі є зв’язність: план будівлі має сенс лише тоді, коли всі поверхи разом утворюють єдиний зв’язний неорієнтований граф. Якщо хоча б одне приміщення або фрагмент коридору не сполучений з рештою графа, побудова маршруту до нього стає неможливою. Тому перевірка зв’язності є обов’язковим етапом підготовки карт.

Додатковою особливістю предметної області є те, що приміщення мають різне функціональне призначення (навчальні аудиторії, лабораторії, кафедри, їдальні, санвузли, адміністративні кабінети), і користувач часто шукає не

конкретну кімнату, а найближче приміщення певного типу. Це зумовлює потребу не лише у пошуку за назвою, а й у фільтруванні за категоріями. Водночас обсяг даних (десятки приміщень на поверсі, кілька поверхів у корпусі) є помірним, що дає змогу зберігати графи поверхів у пам’яті та виконувати пошук маршрутів у режимі, близькому до реального часу.

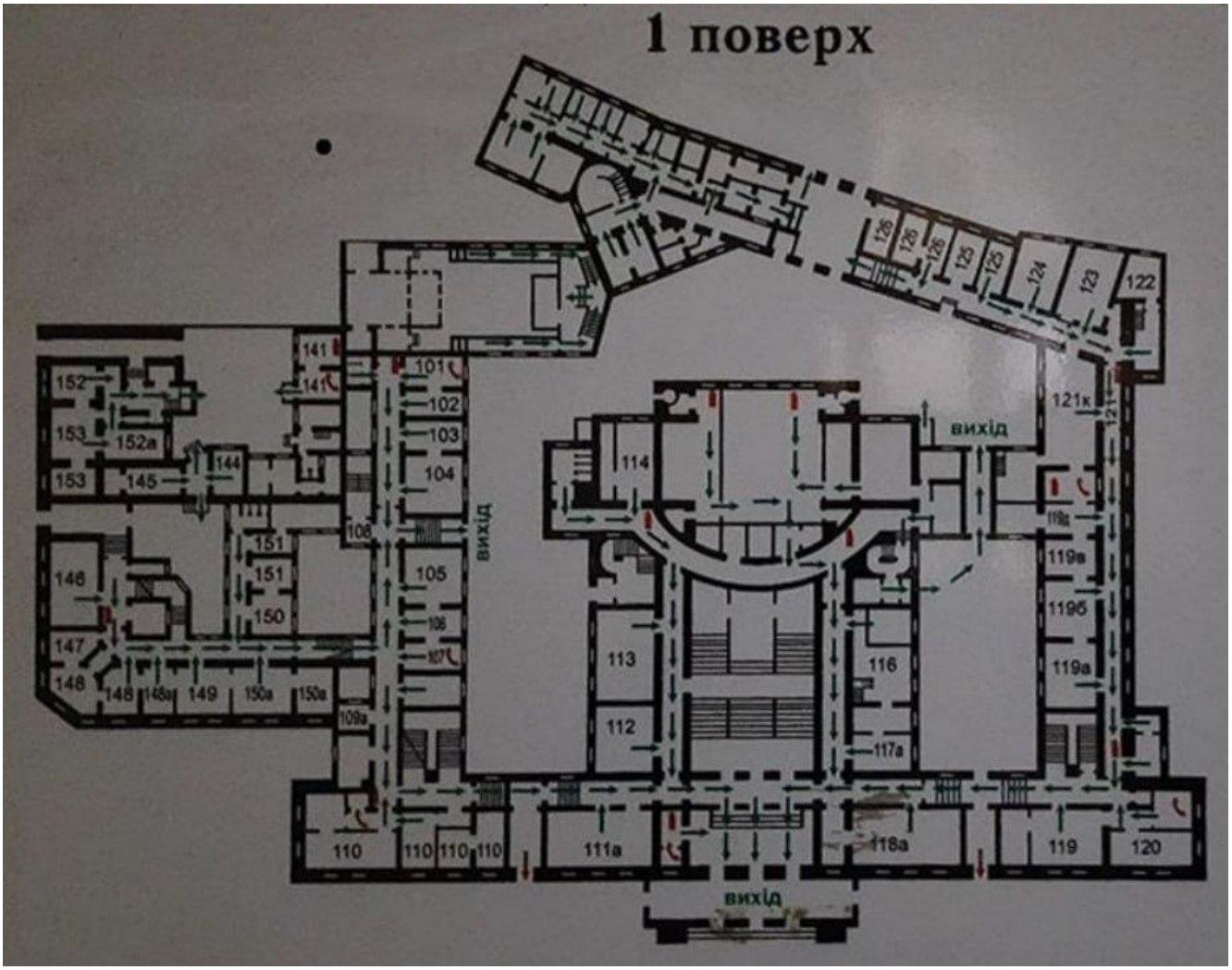


Рисунок 1.1 – Приклад статичного плану поверху, розміщеного при вході до корпусу

1.2 Аналіз наявних рішень

Перед формулюванням вимог було розглянуто наявні підходи до навігації приміщеннями, щоб виявити їхні переваги та недоліки і врахувати їх під час проектування.

1.2.1 Статичні плани та інформаційні стенди

Найпоширенішим рішенням залишаються паперові плани при входах і покажчики на стінах. Їх перевага — нульова вартість для відвідувача та відсутність потреби у пристрої. Проте такі плани статичні: вони не дають змоги виконати пошук конкретної аудиторії, не показують маршрут від поточного місця до цілі, не враховують зміни призначення приміщень і не масштабуються на кілька поверхів чи корпусів. Користувач змушений самостійно зіставляти умовний план із реальним простором, що часто призводить до помилок.

1.2.2 Google Maps та загальні картографічні сервіси

Google Maps підтримує режим Indoor Maps для окремих великих об’єктів (аеропортів, торгових центрів). Сервіс зручний і знайомий користувачам, проте деталізація університетських корпусів зазвичай відсутня або обмежена, а наповнення картографічних даних перебуває поза контролем закладу освіти. Крім того, прокладання маршруту всередині будівлі з урахуванням конкретних аудиторій і вертикальних переходів у загальному сервісі не передбачено.

1.2.3 Спеціалізовані платформи (MazeMap та аналоги)

Існують спеціалізовані комерційні платформи навігації приміщеннями, як-от MazeMap, орієнтовані на університети та лікарні. Вони забезпечують пошук приміщень і побудову маршрутів, проте є пропрієтарними рішеннями зі сплатою за підписку, прив’язкою до інфраструктури постачальника та обмеженими можливостями адаптації під конкретні потреби закладу. Залежність від зовнішнього постачальника ускладнює інтеграцію з внутрішніми системами та контроль над даними.

Проведений аналіз показує, що жодне з розглянутих рішень одночасно не забезпечує: повний контроль закладу над картографічними даними, безкоштовний доступ для відвідувача, пошук приміщень, побудову найкоротшого маршруту з урахуванням переходів між поверхами та сучасний адаптивний веб-інтерфейс. Це обґрунтовує доцільність розробки власного рішення. Порівняння розглянутих підходів за ключовими критеріями наведено у таблиці.

Таблиця 1.1 – Порівняння наявних рішень для навігації приміщеннями

Критерій	**Статичні плани**	**Google Maps**	**MazeMap**	**Unimap**
Пошук приміщень	ні	частково	так	так
Маршрут між поверхами	ні	ні	так	так
Контроль закладу над даними	так	ні	ні	так
Безкоштовний доступ	так	так	ні	так
Адаптивний веб-інтерфейс	ні	так	так	так
Адміністрування карт	ні	ні	обмежено	так

1.3 Функціональні вимоги

Розроблений застосунок задовольняє такі функціональні вимоги:

- перегляд інтерактивного плану обраного корпусу з можливістю масштабування та переміщення (pan/zoom);
- перемикання між поверхами будівлі;
- пошук приміщень за назвою та ключовими словами у межах усього корпусу з підказками під час введення;
- фільтрування приміщень за категоріями (їдальні, лабораторії, санвузли, простори для роботи тощо) з підсвічуванням знайдених кімнат;
- перегляд детальної інформації про приміщення у модальному вікні;
- побудова найкоротшого маршруту між двома приміщеннями, у тому числі між різними поверхами, з відображенням маршруту поверх плану;
- збереження маршрутів і приміщень для швидкого доступу;
- перегляд історії пошуків і побудованих маршрутів;
- перегляд подій університету, прив’язаних до приміщень;
- реєстрація, підтвердження пошти та вхід у систему, а також робота у режимі гостя без реєстрації;

- адміністрування корпусів, поверхів, карт і приміщень через окремий інтерфейс адміністратора.

1.4 Нефункціональні вимоги

До системи висунуто такі нефункціональні вимоги:

- україномовний інтерфейс користувача з можливістю перемикання на англійську мову;
- адаптивність інтерфейсу для настільних і мобільних пристроїв;
- безпека: захищене зберігання паролів, автентифікація на основі токенів, розмежування доступу за ролями;
- цілісність картографічних даних: гарантована зв’язність графа плану будівлі;
- продуктивність: швидка побудова маршруту та плавна взаємодія з картою;
- розширюваність: модульна архітектура, відкрита до додавання нових корпусів, категорій і функцій без переписування системи [4];
- відповідність сучасним практикам зручності використання та доступності інтерфейсу [8], [10].

1.5 Ролі користувачів

У системі передбачено три ролі користувачів:

1) гість — неавторизований користувач, якому доступні перегляд карти, пошук, фільтрування, побудова маршрутів і перегляд подій; для гостя система все ж зберігає окремі дані (наприклад, тимчасову історію);
2) зареєстрований користувач — має всі можливості гостя, а також збереження маршрутів і приміщень та повноцінну персональну історію пошуків і маршрутів;
3) адміністратор — має доступ до інтерфейсу адміністрування: завантаження та оновлення карт, керування корпусами, поверхами, приміщеннями та подіями.

1.6 Формулювання задачі

На підставі проведеного аналізу та сформульованих вимог задачу роботи можна сформулювати так: спроектувати й розробити веб-застосунок інтерактивної навігації приміщеннями університету, який на основі зв’язної графової моделі поверхів забезпечує пошук приміщень, фільтрування за категоріями та побудову найкоротшого маршруту між приміщеннями (зокрема між поверхами), надає сучасний адаптивний двомовний інтерфейс користувача, безпечну автентифікацію з розмежуванням ролей та інструменти адміністрування картографічних даних.

Вимоги до шуканого розв’язку: коректність побудованих маршрутів за умови валідних вхідних карт; гарантована перевірка зв’язності графа будівлі перед публікацією; модульність і розширюваність програмної архітектури; відповідність інтерфейсу принципам зручності використання.

Окремо слід наголосити, що задача поділяється на дві принципово різні за природою частини. Перша — підготовка та валідація просторових даних: вона виконується нечасто (під час додавання чи оновлення корпусу) і вимагає суворого контролю якості, адже одна некоректна карта здатна зробити навігацію хибною для всіх користувачів. Друга — обслуговування запитів кінцевих користувачів: вона виконується постійно й масово, тому до неї висуваються вимоги швидкодії та стабільності. Такий поділ безпосередньо впливає на архітектуру системи, описану в розділі 3.

1.7 Припущення та обмеження

Під час проектування системи прийнято низку припущень, що окреслюють межі застосовності розв’язку та спрощують подальші рішення:

- вихідні плани поверхів надає адміністратор у заздалегідь узгодженому форматі з коректно розміченими приміщеннями, вузлами та ребрами;
- геометрія планів подана у єдиній системі координат у межах одного поверху, що дає змогу обчислювати відстані між вузлами;
- вертикальні переходи (сходи) присутні на всіх поверхах, які вони сполучають, і мають узгоджені ідентифікатори;

- користувач має доступ до сучасного браузера з підтримкою стандартів HTML5 та SVG;
- масштаб закладу обмежується кількома корпусами, що дає змогу зберігати графи поверхів у пам’яті та кешувати їх для пришвидшення обчислень.

Зазначені припущення не звужують практичної цінності продукту, проте чітко визначають умови, за яких гарантується коректність його роботи, і слугують орієнтиром для подальшого розгортання системи в нових корпусах.

РОЗДІЛ 2
ВИКОРИСТАНИЙ НАБІР ТЕХНОЛОГІЙ ТА МЕТОДІВ

Для реалізації застосунку обрано сучасний набір технологій екосистеми JavaScript/TypeScript, що дає змогу використовувати єдину мову програмування як на сервері, так і на клієнті, спрощує повторне використання типів і знижує когнітивне навантаження на розробника. Вибір кожної складової підпорядковано вимогам, сформульованим у розділі 1: безпеці, продуктивності, цілісності даних і розширюваності. Нижче обґрунтовано кожне технологічне рішення.

2.1 TypeScript

Як основну мову програмування для всіх частин системи обрано TypeScript — надбудову над JavaScript зі статичною типізацією [28]. Статична типізація дає змогу виявляти значну частину помилок ще на етапі компіляції, покращує автодоповнення та робить код самодокументованим. У проєкті використано TypeScript версії 5.7 у суворому режимі (strict), що максимально посилює перевірки типів.

Окремою перевагою TypeScript у контексті повностекового застосунку є можливість описувати форму даних один раз і використовувати ці типи як на сервері, так і на клієнті. Це усуває цілий клас помилок, пов’язаних із неузгодженістю структур даних між частинами системи, та робить рефакторинг безпечнішим: зміна типу одразу виявляє всі місця коду, які потребують оновлення. Розвинена система узагальнених типів (generics) дає змогу будувати гнучкі, але водночас типобезпечні абстракції.

2.2 Node.js

Серверне середовище виконання — Node.js [21], платформа на основі рушія V8 з неблокувальною подієво-орієнтованою моделлю введення-виведення, що добре пасує для мережевих застосунків з великою кількістю одночасних запитів. Node.js забезпечує доступ до широкої екосистеми пакетів npm та підтримує сучасний синтаксис ECMAScript.

Однопотокова, але асинхронна модель Node.js особливо ефективна для задач, у яких переважають операції очікування (звернення до бази даних, сховища, поштового сервера), що характерно для навігаційного застосунку. Використано довгострокову (LTS) версію середовища, яка гарантує тривалу підтримку та стабільність, що важливо для експлуатації продукту.

2.3 NestJS та Fastify

Серверний каркас побудовано на NestJS версії 11 [19] — прогресивному фреймворку для Node.js, що заохочує модульну, об'єктно-орієнтовану архітектуру з впровадженням залежностей (dependency injection). Такий підхід природно поєднується з принципами доменно-орієнтованого проектування [4] та полегшує дотримання принципів SOLID.

Впровадження залежностей робить компоненти слабко зв'язаними та легко тестованими: реальні реалізації сервісів можна підмінювати на тестові двійники без зміни коду, що залежить від них. Як HTTP-адаптер замість стандартного Express використано Fastify [15] — високопродуктивний веб-сервер з низькими накладними витратами та вбудованою валідацією схем, що позитивно впливає на пропускну здатність прикладного інтерфейсу.

2.4 PostgreSQL та Drizzle ORM

Дані зберігаються у реляційній системі керування базами даних PostgreSQL [22], що відома надійністю, підтримкою стандарту SQL, ACID-транзакцій, типу JSONB та повнотекстового і триграмного пошуку. Транзакційність є критично важливою для узгодженого оновлення карт і приміщень, а підтримка JSONB дає змогу зберігати граф поверху як структурований документ поряд із реляційними даними.

Для доступу до бази даних використано Drizzle ORM версії 0.45 [14] — типобезпечний інструмент, що дає змогу описувати схему та запити мовою TypeScript і автоматично генерувати міграції, зберігаючи близькість до SQL. На відміну від важких об'єктно-реляційних відображень, Drizzle не приховує SQL, а робить його типобезпечним: структура таблиць є єдиним джерелом істини, з якого

виводяться як типи даних, так і файли міграцій, що усуває розбіжності між кодом і фактичною схемою бази.

2.5 GraphQL (Mercurius) та REST

Прикладний програмний інтерфейс системи побудовано у двох взаємодоповнювальних стилях. Основні ресурсно-орієнтовані операції (карти, кімнати, навігація, автентифікація) надано через REST поверх Fastify, теоретичні засади якого сформульовано у роботі Р. Філдінга [5]. REST зручний для простих, кешованих ресурсних запитів і добре узгоджується з інфраструктурою вебу.

Паралельно реалізовано GraphQL-інтерфейс на основі адаптера Mercurius версії 16 та пакета @nestjs/graphql, що відповідає офіційній специфікації мови запитів GraphQL [16], [1]. GraphQL дає клієнтові змогу запитувати рівно ті дані, яких він потребує, що особливо зручно для агрегації відомостей про корпуси, поверхи, приміщення та події в одному запиті й усуває проблеми надмірного та недостатнього вибирання даних, притаманні класичному REST. Схема GraphQL охоплює запити та мутації для карт, кімнат, пошуку маршрутів, автентифікації та подій.

2.6 Автентифікація: JWT та bcrypt

Автентифікацію реалізовано на основі веб-токенів JSON Web Token відповідно до стандарту RFC 7519 [17]. Маркери доступу та оновлення передаються у захищених cookie. Корисне навантаження токена доступу містить ідентифікатор користувача, його роль та ідентифікатор сесії, що дає змогу авторизувати запити без звернення до бази даних на кожному кроці.

Паролі користувачів зберігаються лише у вигляді хеш-значень, обчислених адаптивним алгоритмом bcrypt [11], що забезпечує стійкість до перебору завдяки регульованому коефіцієнту складності обчислення. Адаптивність алгоритму означає, що зі зростанням обчислювальної потужності зловмисників складність можна підвищувати без зміни архітектури, що робить рішення стійким у довгостроковій перспективі.

2.7 Redis та BullMQ

Для фонового опрацювання завдань, насамперед надсилання листів підтвердження пошти, використано чергу повідомлень BullMQ [13], що працює поверх сховища Redis [25]. Винесення повільних операцій у фонову чергу робить відповідь сервера швидшою та підвищує надійність доставлення листів: завдання повторюються в разі тимчасових збоїв, а їх стан зберігається у Redis і не втрачається при перезапуску процесу.

2.8 Об’єктне сховище (S3-сумісне)

Вихідні файли карт (SVG) зберігаються в об’єктному сховищі, сумісному з протоколом Amazon S3. Сервер видає клієнтові тимчасові підписані посилання на файли, що дає змогу віддавати великі статичні ресурси безпосередньо зі сховища, не навантажуючи прикладний сервер. Підписані посилання мають обмежений строк дії, тому доступ до файлів залишається контрольованим, попри пряме завантаження з боку клієнта.

2.9 Next.js та React

Клієнтську частину побудовано на бібліотеці React версії 19 [23] та каркасі Next.js версії 15 [20] з використанням маршрутизатора застосунку (App Router) і серверних компонентів. React забезпечує компонентний підхід до побудови інтерфейсу із застосуванням сучасних патернів розробки [2], а Next.js додає маршрутизацію, оптимізацію та зручну організацію коду.

Серверні компоненти Next.js дають змогу виконувати частину рендерингу на сервері, зменшуючи обсяг коду, що завантажується у браузер, і пришвидшуючи перше відображення сторінки. Вбудовані механізми розділення коду, оптимізації зображень і маршрутизації за файловою структурою спрощують створення продуктивного застосунку без значного обсягу конфігурації.

2.10 Tailwind CSS та Material Design 3

Для стилізації застосовано utility-first фреймворк Tailwind CSS версії 4 [27]. Замість написання окремих каскадних таблиць стилів інтерфейс збирається з

невеликих службових класів безпосередньо у розмітці, що пришвидшує розробку та запобігає розростанню глобальних стилів.

Візуальну мову інтерфейсу побудовано на дизайн-системі Material Design 3 [18]: її семантичні токени кольору, типографіки та тіней відображено у CSS-змінних і підключено до Tailwind, що забезпечує узгоджений вигляд та підтримку світлої й темної тем. Використання усталеної дизайн-системи підвищує впізнаваність елементів керування та зменшує час адаптації користувача.

2.11 Керування станом: React Query та Zustand

Стан клієнта поділено на дві категорії. Серверний стан (дані з прикладного інтерфейсу — корпуси, приміщення, історія) кешується та синхронізується бібліотекою TanStack Query (React Query) версії 5 [24], яка бере на себе кешування, фонове оновлення, повторні спроби та інвалідацію даних.

Локальний стан інтерфейсу (вибраний поверх, активний інструмент, підсвічені кімнати, відкриті діалоги, сфокусоване приміщення) зберігається у легкому сховищі Zustand версії 5 [30]. Чіткий поділ між серверним і клієнтським станом запобігає поширеній помилці — дублюванню серверних даних у глобальному сховищі — і робить логіку передбачуванішою та простішою для супроводу.

2.12 Допоміжні бібліотеки

Окрім основних, у проєкті використано низку допоміжних бібліотек:

- Zod — декларативна валідація та виведення типів зі схем, застосована для перевірки змінних середовища, тіл запитів і форм [29];
- jsdom — серверна реалізація DOM, що використовується для безпечного розбору SVG-карт відповідно до специфікації SVG [26];
- react-zoom-pan-pinch — компонент масштабування та переміщення для інтерактивної карти на клієнті;
- react-hook-form — продуктивне керування формами входу й реєстрації;
- axios — HTTP-клієнт із перехопленням відповідей для автоматичного оновлення токенів;

- next-intl — інтернаціоналізація з маршрутизацією за локаллю;
- sonner — спливні сповіщення (toast) у клієнтському інтерфейсі.

2.13 Засоби тестування

Якість коду забезпечують автоматизовані тести. Модульні та інтеграційні тести серверної частини написано з використанням фреймворку Jest, а перевірка типів виконується компілятором TypeScript [28]. Тести охоплюють бізнес-логіку сервісів, контролери та взаємодію модулів між собою; докладніше їх описано у підрозділі 4.13.

2.14 Інструменти забезпечення якості та збирання

Єдність стилю коду підтримують статичний аналізатор ESLint та форматувальник Prettier, налаштовані спільно для обох робочих просторів монорепозиторію. Це усуває суперечки щодо оформлення коду та автоматично виявляє низку типових помилок ще до запуску програми. Керування залежностями та сценаріями збирання здійснює пакетний менеджер npm з механізмом робочих просторів (workspaces), що дає змогу встановлювати залежності одночасно для серверної та клієнтської частин і спільно використовувати конфігурацію TypeScript [28].

Змінні середовища (рядки підключення до бази даних, секрети токенів, параметри об’єктного сховища та поштового сервера) перевіряються на старті застосунку схемами Zod [29]. Якщо обов’язкова змінна відсутня або має некоректне значення, застосунок не запускається, повідомляючи про проблему, що запобігає роботі системи у некоректній конфігурації.

2.15 Алгоритм для пошуку найкоротшого шляху на графі A*

Ключовим алгоритмічним методом, застосованим у роботі, є алгоритм пошуку найкоротшого шляху A* (A-star). Оскільки план кожного поверху подається у вигляді зваженого неорієнтованого графа, задача побудови маршруту між двома приміщеннями зводиться до класичної задачі пошуку найкоротшого

шляху у графі [3]. Для її розв’язання обрано інформований (евристичний) алгоритм A*, уперше запропонований П. Гартом, Н. Нільссоном і Б. Рафаелем [6].

Алгоритм A* поєднує переваги алгоритму Дейкстри та жадібного пошуку за першим найкращим збігом: на кожному кроці він розширює вершину з найменшим значенням оцінювальної функції $f(n) = g(n) + h(n)$, де $g(n)$ — фактична вартість шляху від початкової вершини до поточної вершини n, а $h(n)$ — евристична оцінка вартості шляху від n до цільової вершини. Як евристику використано евклідову відстань між координатами вузлів, яка є допустимою (не переоцінює реальну відстань). За умови допустимості евристики алгоритм A* гарантовано знаходить оптимальний (найкоротший) маршрут [12].

Перевага A* над неінформованим пошуком Дейкстри полягає в тому, що евристика спрямовує пошук у бік цілі й суттєво зменшує кількість переглянутих вершин, пришвидшуючи побудову маршруту. Якщо евристична оцінка дорівнює нулю (наприклад, коли координати вузла невідомі), алгоритм коректно вироджується у пошук Дейкстри. Для ефективного вибору наступної вершини множину відкритих вершин реалізовано на основі двійкової купи (мінімальної піраміди) за ключем $f(n)$ [3]. Деталі програмної реалізації алгоритму, зокрема обчислення евристики та об’єднання графів окремих поверхів у єдиний граф будівлі, наведено у підрозділі 4.6.

РОЗДІЛ 3
ПРОЕКТУВАННЯ СИСТЕМИ ТА МОДЕЛЬ ДАНИХ

3.1 Загальна архітектура системи

Систему побудовано за клієнт-серверною архітектурою. Серверна частина (застосунок API) надає прикладний програмний інтерфейс і відповідає за бізнес-логіку, доступ до даних і безпеку. Клієнтська частина (вебзастосунок) виконується у браузері користувача та відповідає за відображення інтерактивної карти й взаємодію з користувачем. Обидві частини організовано як монорепозиторій з робочими просторами npm (каталоги apps/api та apps/web), що спрощує спільне використання конфігурації і типів.

Серверну частину спроектовано за принципами доменно-орієнтованого проектування [4]: предметну область поділено на обмежені контексти (bounded contexts), кожен з яких відповідає за власну ділянку логіки. Такий поділ забезпечує слабку зв’язність між модулями та високу зв’язність усередині модуля, що відповідає принципам SOLID і робить систему відкритою до розширення. Виокремлено такі модулі: автентифікація (auth), користувачі (users), карти (maps), приміщення (rooms), пошук шляхів (wayfinding), історія (history), події (events), сповіщення (notifications) та службовий модуль стану (health).

Прикладний інтерфейс реалізовано у стилі REST, теоретичні засади якого сформульовано у роботі Р. Філдінга [5], та доповнено інтерфейсом GraphQL. Загальну архітектуру системи показано на рисунку, що відображає взаємодію клієнта, прикладного сервера, бази даних, об’єктного сховища та черги повідомлень.

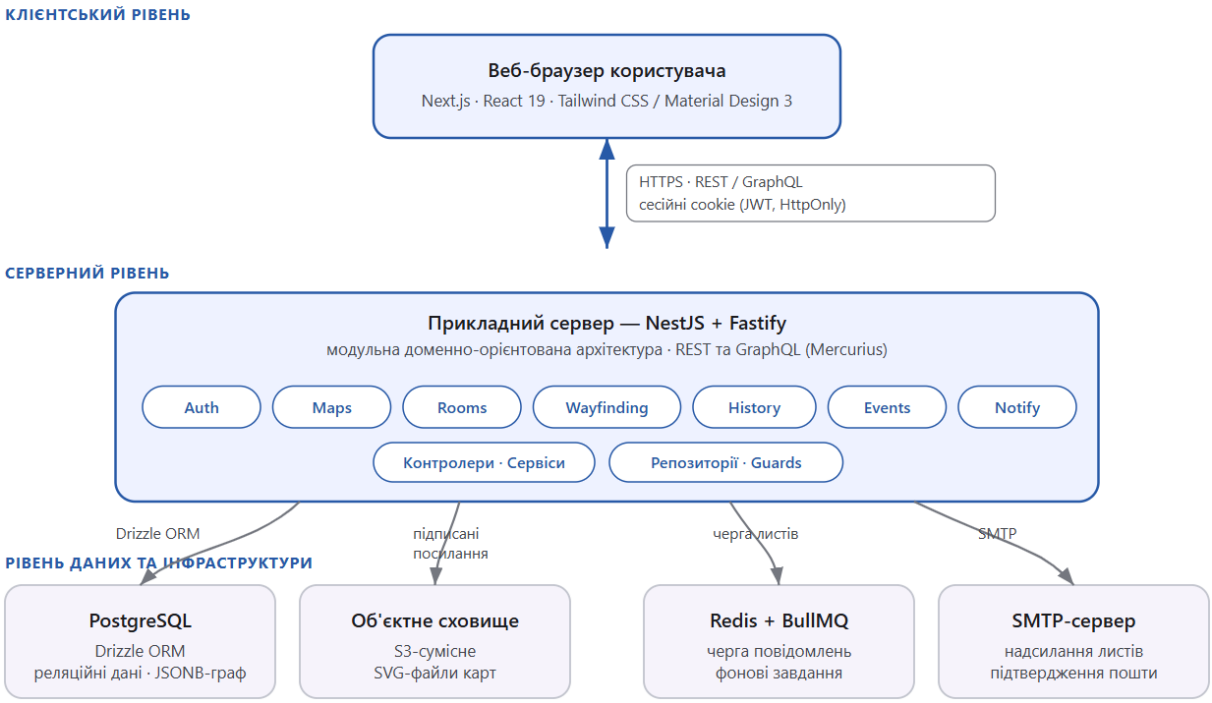


Рисунок 3.1 – Загальна архітектура системи Unimap

3.2 Модель даних

Реляційну модель даних спроектовано у PostgreSQL [22]. Топологічні дані графа (вузли та ребра) зберігаються у форматі JSONB у складі версіонованого ресурсу карти, тоді як відомості про приміщення денормалізовано в окрему таблицю для швидкого пошуку. Основні сутності та їхні зв’язки наведено нижче.

- users — користувачі: ідентифікатор, електронна пошта, хеш пароля, роль (guest, user, admin), ознака анонімності, час підтвердження пошти;
- sessions — сесії оновлення: користувач, ідентифікатор родини токенів, хеш токена оновлення, відомості про пристрій, час завершення та відкликання;
- email_verification_tokens — токени підтвердження електронної пошти;
- buildings — корпуси: унікальний код, назва, адреса, статус (draft, live);
- floors — поверхи: належність до корпусу, номер, назва, посилання на активний ресурс карти;
- map_assets — версіоновані ресурси карт: SVG-файл у сховищі, контрольна сума, кількість вузлів, ребер і кімнат та граф у форматі JSONB;

- rooms — приміщення: код, поверх, назва, категорія, ключові слова, ознака доступності, первинний вузол навігації та пошуковий індекс;
- saved_items — збережені користувачем приміщення та маршрути;
- events — події університету, прив’язані до приміщень і часу;
- room_search_history та path_history — історія пошуків приміщень і побудованих маршрутів відповідного користувача.

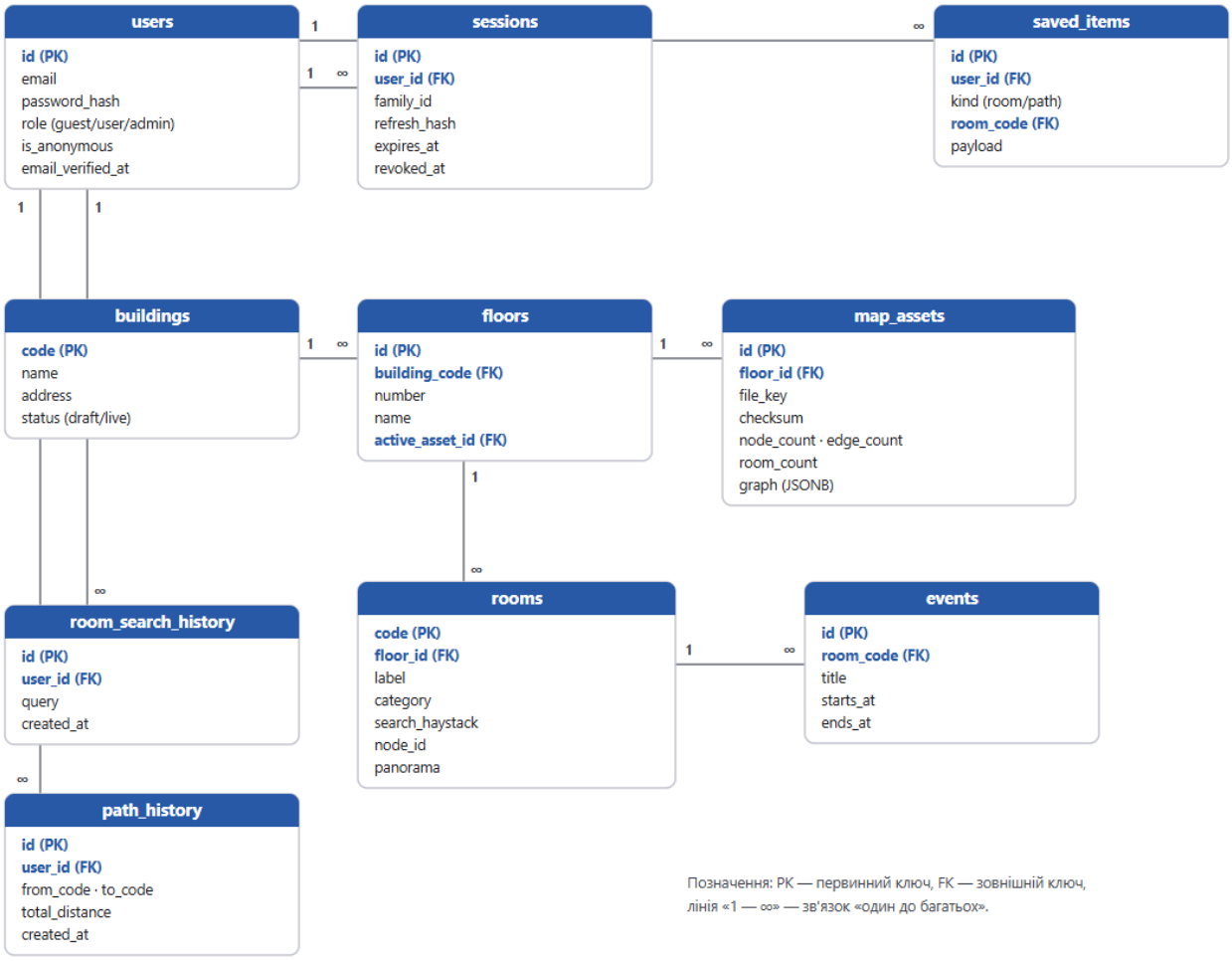


Рисунок 3.2 – Діаграма «сутність — зв’язок» бази даних

Зв’язки між основними сутностями підпорядковано доменній логіці: корпус містить кілька поверхів, поверх посилається на активну версію ресурсу карти, а приміщення належить до поверху та має первинний вузол навігації. Призначення ключових таблиць узагальнено у таблиці нижче.

Таблиця 3.1 – Призначення основних таблиць бази даних

Таблиця	Призначення
users	облікові записи користувачів, ролі та стан підтвердження пошти
sessions	сесії оновлення з підтримкою ротації та відкликання
buildings	корпуси університету та їхній статус публікації
floors	поверхи корпусів і посилання на активну карту
map_assets	версіоновані карти: SVG, контрольна сума, граф (JSONB)
rooms	приміщення: код, назва, категорія, пошуковий індекс
events	події університету, прив’язані до приміщень і часу
saved_items	збережені користувачем приміщення та маршрути
*_history	історія пошуків приміщень і побудованих маршрутів

3.3 Модель плану поверху як графа

Центральним проєктним рішенням є подання плану поверху у вигляді зваженого неорієнтованого графа. Вхідним форматом карти обрано SVG — відкритий векторний формат W3C [26], який одночасно є придатним для відображення та містить структуровані метадані. Кожен елемент плану позначено атрибутом data-name:

- g[data-name="room"] — приміщення; має ідентифікатор-код, назву (data-label), категорію (data-category), ключові слова та посилання на первинний вузол навігації (data-node-id);
- g[data-name="node"] — вузол графа (вхід до приміщення, перехрестя коридору, сходовий майданчик); має ідентифікатор і тип (звичайний, сходи), координати беруться з геометрії дочірньої фігури;
- g[data-name="edge"] — ребро графа; задає пару з’єднаних вузлів через атрибут data-nodes-id та необов’язкову вагу (data-weight).

Вага ребра відповідає довжині відрізка шляху, що дає змогу будувати географічно правдоподібні маршрути. Перехід між поверхами моделюється спеціальними міжповерховими вузлами (сходами): такі вузли позначаються службовим токеном поверху «00» і присутні на планах кількох поверхів з

однаковим ідентифікатором, завдяки чому графи окремих поверхів об’єднуються в єдиний граф будівлі.

```
239         </g>
240         <g id="edges">
241             <g id="LWPOLYLINE-56" data-name="LWPOLYLINE">
242                 <polygon class="cls-3"
243                     points="244.89 511.91 241.5 511.91 241.5 502.95 244.89 502.95 244.89 511.91 244.89 511.91" />
244             </g>
245             <g id="LWPOLYLINE-57" data-name="LWPOLYLINE">
246                 <polygon class="cls-3"
247                     points="232.54 474.44 232.54 471.06 241.5 471.06 241.5 474.44 232.54 474.44 232.54 474.44" />
248             </g>
249             <g id="LWPOLYLINE-58" data-name="LWPOLYLINE">
250                 <polygon class="cls-3"
251                     points="232.54 386.47 232.54 389.85 241.5 389.85 241.5 386.47 232.54 386.47 232.54 386.47" />
252             </g>
253             <g id="LWPOLYLINE-59" data-name="LWPOLYLINE">
254                 <path class="cls-3"
255                     d="M350.11,352l-3.43-6.8a109,109,0,0,1-32.87,10.49V332.89h3.38v18.82a105.86,105.86,0,0,0,31-11.
256             </g>
257             <g id="LWPOLYLINE-60" data-name="LWPOLYLINE">
258                 <polygon class="cls-3"
259                     points="322.81 329.51 322.81 332.9 317.19 332.9 317.19 329.51 322.81 329.51 322.81 329.51" />
260             </g>
261             <g id="LWPOLYLINE-61" data-name="LWPOLYLINE">
262                 <polygon class="cls-3"
263                     points="353.5 511.91 350.11 511.91 350.11 502.95 353.5 502.95 353.5 511.91 353.5 511.91" />
```

```
699         </g>
700         <g id="nodes">
701             <g id="LWPOLYLINE-183" data-name="LWPOLYLINE">
702                 <rect class="cls-4" x="540.27" y="511.91" width="3.38" height="3.38" />
703             </g>
704             <g id="LWPOLYLINE-184" data-name="LWPOLYLINE">
705                 <rect class="cls-4" x="521.15" y="511.91" width="3.38" height="3.38" />
706             </g>
707             <g id="LWPOLYLINE-185" data-name="LWPOLYLINE">
708                 <rect class="cls-4" x="401.61" y="511.91" width="3.38" height="3.38" />
709             </g>
710             <g id="LWPOLYLINE-186" data-name="LWPOLYLINE">
711                 <rect class="cls-4" x="350.11" y="511.91" width="3.38" height="3.38" />
712             </g>
713             <g id="LWPOLYLINE-187" data-name="LWPOLYLINE">
714                 <rect class="cls-4" x="241.5" y="511.91" width="3.38" height="3.38" />
715             </g>
716             <g id="LWPOLYLINE-188" data-name="LWPOLYLINE">
717                 <rect class="cls-4" x="190" y="511.91" width="3.38" height="3.38" />
718             </g>
719             <g id="LWPOLYLINE-189" data-name="LWPOLYLINE">
720                 <rect class="cls-4" x="70.46" y="511.91" width="3.38" height="3.38" />
721             </g>
722             <g id="LWPOLYLINE-190" data-name="LWPOLYLINE">
723                 <rect class="cls-4" x="51.35" y="511.91" width="3.38" height="3.38" />
724             </g>
```

Рисунок 3.3 – Фрагмент SVG-плану та відповідний граф вузлів і ребер

Розглянемо приклад. Нехай на десятому поверсі десятого корпусу аудиторія з кодом 10-01 має первинний вузол навігації при вході; цей вузол сполучено ребром

із вузлом коридору, який, своєю чергою, через послідовність ребер веде до сходового вузла. Сходовий вузол позначено як міжповерховий за допомогою токена «00» і присутній на планах сусідніх поверхів з тим самим ідентифікатором. Під час злиття графів цей спільний вузол стає «мостом» між поверхами, завдяки чому алгоритм пошуку шляху безперешкодно переходить з поверху на поверх, розглядаючи багатоповерховий маршрут як шлях в єдиному графі будівлі.

Такий підхід має суттєву перевагу: рендеринг карти й побудова маршруту використовують одне й те саме джерело даних — розмічений SVG. Це усуває потребу в окремому форматі для геометрії та для топології й гарантує, що відображувана карта та граф маршрутизації завжди узгоджені між собою.

3.4 Перевірка цілісності та зв’язності карт

Оскільки коректність навігації повністю залежить від якості вхідних карт, у системі передбачено багаторівневу валідацію. На рівні окремого поверху перевіряється відповідність ідентифікаторів прийнятим конвенціям, існування кінцевих вузлів кожного ребра, відсутність петель, а також зв’язність усіх вузлів поверху за допомогою обходу в ширину (BFS) [3].

На рівні будівлі графи всіх поверхів об’єднуються в один неорієнтований граф, після чого повторним обходом у ширину перевіряється, що він утворює єдину зв’язну компоненту. Публікація карти організована як життєвий цикл «чернетка → фіналізація → опублікована» (draft → finalize → live): корпус стає доступним користувачам лише після успішної перевірки глобальної зв’язності. Якщо граф розпадається на кілька компонент, фіналізацію відхиляють із зазначенням проблемних вузлів, що унеможливлює публікацію некоректної карти.

3.5 Варіанти використання

Відповідно до визначених у підрозділі 1.5 ролей побудовано діаграму варіантів використання. Гість виконує перегляд карти, пошук, фільтрування, побудову маршрутів і перегляд подій. Зареєстрований користувач додатково зберігає приміщення й маршрути та переглядає персональну історію. Адміністратор керує корпусами, картами, приміщеннями та подіями.

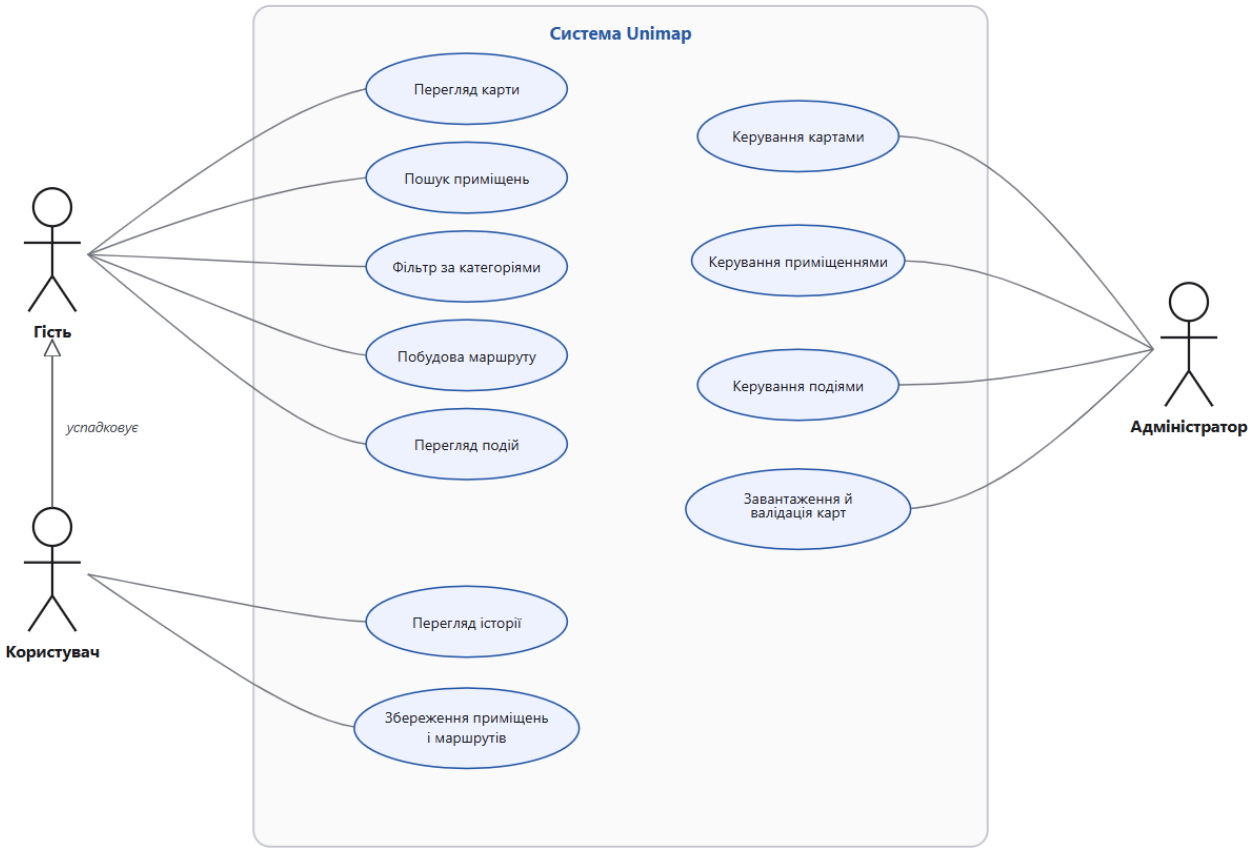


Рисунок 3.4 – Діаграма варіантів використання системи

3.6 Діаграми послідовностей основних процесів

Для уточнення динаміки системи розглянуто діаграми послідовностей ключових процесів. Процес автентифікації охоплює введення облікових даних, перевірку їх сервером, видачу токенів доступу й оновлення та встановлення захищених cookie.

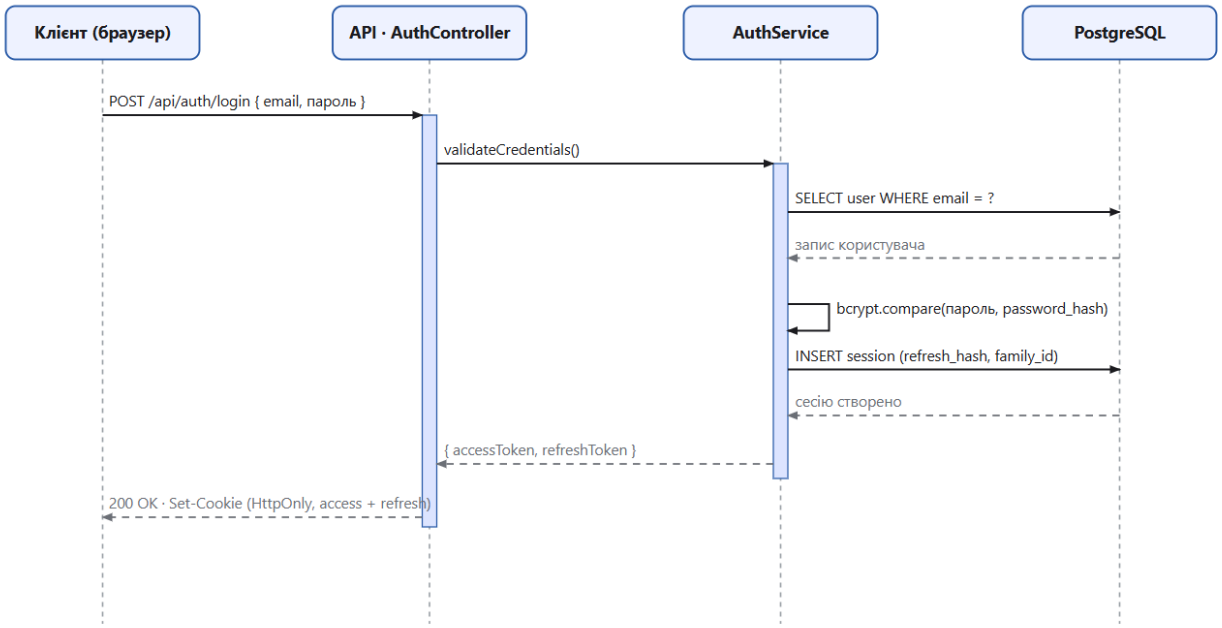


Рисунок 3.5 – Діаграма послідовності процесу автентифікації

Процес пошуку приміщення охоплює введення запиту, надсилання його з урахуванням затримки (debounce), повнотекстовий пошук на сервері та повернення підказок, після вибору яких карта фокусується на знайденому приміщенні.

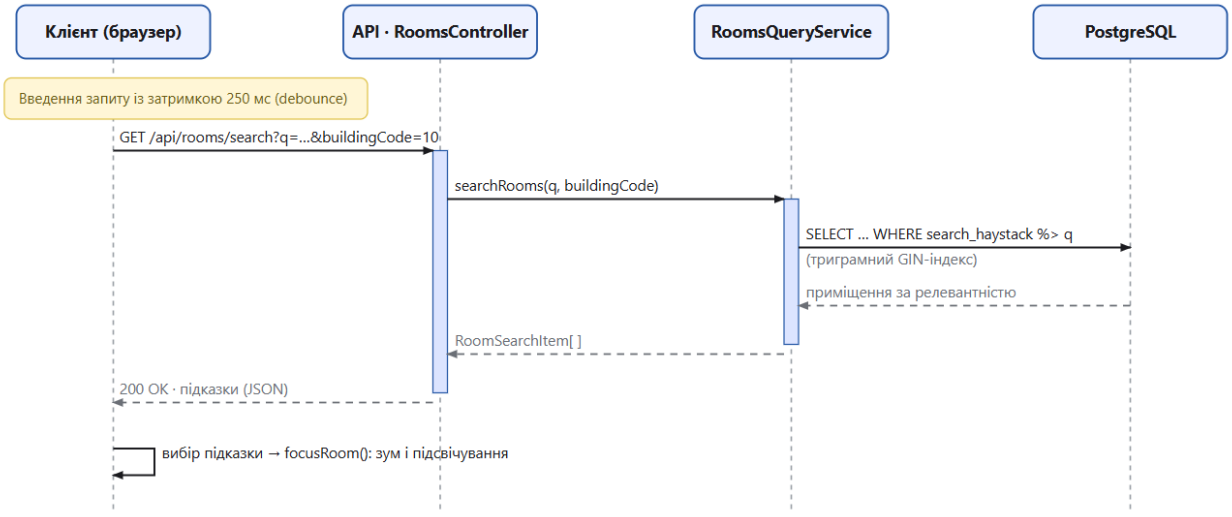


Рисунок 3.6 – Діаграма послідовності пошуку приміщення

Процес побудови маршруту охоплює вибір початкового та кінцевого приміщень, звернення до сервера, виконання алгоритму пошуку найкоротшого шляху та відображення маршруту поверх плану з можливим перемиканням поверхів.

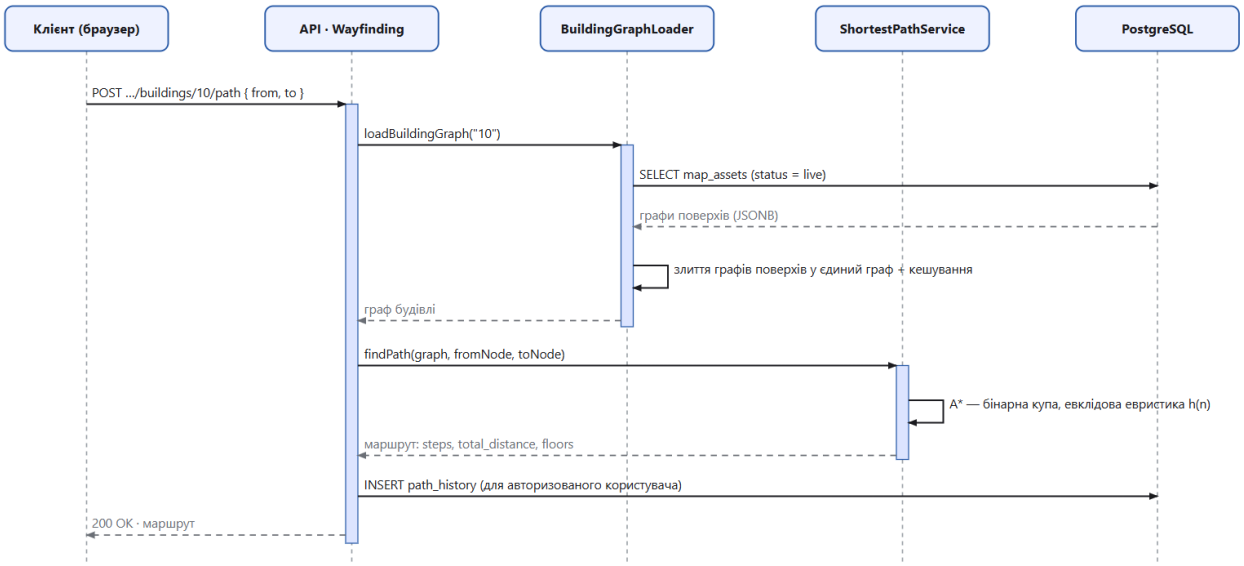


Рисунок 3.7 – Діаграма послідовності побудови маршруту

3.7 Версіонування карт та узгодженість даних

Карти поверхів змінюються з часом: перейменовуються аудиторії, перебудовуються коридори, додаються нові приміщення. Щоб ці зміни не порушували роботу системи, ресурси карт версіоновано. Кожне завантаження SVG-файлу створює новий запис у таблиці map_assets, а поверх посилається на активну версію карти. Завдяки цьому можна зберігати історію змін, порівнювати версії та за потреби повертатися до попередньої без втрати даних.

Узгодженість денормалізованих даних про приміщення підтримується синхронізацією: після активації нової версії карти таблиця приміщень оновлюється відповідно до вмісту графа в межах однієї транзакції бази даних [22]. Транзакційність гарантує, що користувачі ніколи не побачать часткового або суперечливого стану карти.

3.8 Безпека та розмежування доступу на рівні проектування

Безпеку закладено в архітектуру на етапі проектування. Доступ до операцій розмежовано за ролями: публічні операції (перегляд карти, пошук, маршрути) доступні всім, персональні (історія, збережене) — лише автентифікованим користувачам, а адміністративні — лише ролі адміністратора. Облікові дані ніколи не зберігаються у відкритому вигляді: паролі — лише у формі адаптивних хешів [11], маркери оновлення — у формі хешів із підтримкою відкликання [17]. Усі вхідні дані проходять валідацію за схемами, що запобігає некоректним і зловмисним запитам.

РОЗДІЛ 4
ОСОБЛИВОСТІ ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

4.1 Структура серверної частини

Серверну частину реалізовано як набір модулів NestJS, кожен з яких відповідає обмеженому контексту предметної області. Усередині модуля застосовано шарувату структуру: контролери (controllers) приймають HTTP- або GraphQL-запити, сервіси (services) містять бізнес-логіку, репозиторії (repositories) інкапсулюють доступ до бази даних, а об’єкти передавання даних (DTO) описують форму вхідних і вихідних даних. Такий поділ відповідальностей знижує зв’язність і полегшує супровід та розширення системи [4], [9].

```
apps/api/src/
├── main.ts                    # точка входу (Fastify)
├── app.module.ts              # кореневий модуль
├── core/                      # спільна інфраструктура
│   ├── auth/                  # guards, декоратори ролей
│   ├── config/                # env.schema.ts (Zod)
│   ├── database/              # Drizzle ORM · schema/
│   └── storage/               # S3-сумісне сховище
└── modules/                   # обмежені контексти (DDD)
    ├── auth/
    │   ├── controllers/       # auth.controller.ts
    │   ├── services/          # auth · token · session
    │   ├── repositories/      # sessions · email-tokens
    │   └── auth.module.ts
    ├── users/                 # облікові записи, ролі
    ├── maps/
    │   ├── controllers/       # maps · admin-maps
    │   ├── services/          # query · admin · validation
    │   ├── parsers/           # svg-floor-parser (jsdom)
    │   ├── repositories/      # buildings · floors · assets
    │   └── maps.module.ts
    ├── rooms/                 # пошук · категорії · панорами
    ├── wayfinding/            # shortest-path (A*) · binary-heap
    ├── history/               # історія пошуків і маршрутів
    ├── events/                # події університету
    ├── notifications/         # email.processor (BullMQ)
    └── health/                # контролер та resolver стану
```

Рисунок 4.1 – Структура модулів серверної частини

4.2 Прикладний інтерфейс: REST та GraphQL

Ресурсно-орієнтовані операції надано через REST під префіксом /api. Основні групи кінцевих точок наведено нижче.

- /api/auth — реєстрація, вхід, гостьовий вхід, підтвердження пошти, оновлення токенів, вихід, отримання профілю;
- /api/maps — перелік корпусів, відомості про корпус, граф поверху та видача SVG-файлу через підписане посилання;
- /api/rooms — категорії, пошук приміщень, детальні відомості про приміщення;
- /api/maps/buildings/:code/path — побудова найкоротшого маршруту між двома приміщеннями;
- /api/history — історія пошуків і маршрутів користувача;
- /api/events — перелік і деталі подій університету;
- /api/saved — збережені приміщення та маршрути;
- /api/admin/* — операції адміністрування карт, корпусів, приміщень і подій.

Паралельно функціонує інтерфейс GraphQL за адресою /graphql, реалізований адаптером Mercurius [19]. Схема охоплює запити для отримання корпусів, поверхів, приміщень, подій та результатів пошуку, а також мутації для автентифікації та адміністрування. GraphQL дає клієнтові змогу одним запитом отримати зв’язані дані (наприклад, корпус разом із поверхами та подіями), уникаючи надлишкових звернень до сервера [1].

Фрагмент схеми GraphQL, що описує тип корпусу та запит для його отримання разом із поверхами й подіями, наведено нижче.

```
type Building {
  code: ID!
  name: String!
  floors: [Floor!]!
  events: [Event!]!
}

type Query {
  building(code: ID!): Building
  searchRooms(buildingCode: ID!, query: String!): [Room!]!
  findPath(buildingCode: ID!, from: ID!, to: ID!): Path!
}
```

Рисунок 4.2 – Фрагмент схеми GraphQL

4.3 Автентифікація та авторизація

Автентифікація ґрунтується на парі веб-токенів JWT [17]: короткоживучому маркері доступу (access token, типовий строк дії 15 хвилин) і довгоживучому маркері оновлення (refresh token, 30 діб). Обидва передаються у захищених HttpOnly cookie, що убезпечує їх від доступу зі сценаріїв на стороні клієнта. Корисне навантаження маркера доступу містить ідентифікатор користувача, його роль і ідентифікатор сесії.

Маркери оновлення зберігаються у базі даних у вигляді bcrypt-хешів [11] і об’єднуються у «родини» (families). Під час кожного оновлення токена стара сесія відкликається й видається нова (ротація). Якщо система виявляє повторне використання вже відкликаного маркера оновлення, вона вважає це ознакою компрометації та відкликає всю родину сесій, що є усталеною практикою захисту від крадіжки токенів.

Авторизацію реалізовано глобальними стражами (guards): страж автентифікації зчитує маркер доступу з cookie, а страж ролей перевіряє відповідність ролі користувача вимогам кінцевої точки. Декоратори @Public, @AllowGuest і @Roles дають змогу декларативно описувати політику доступу. Гість представлений у базі даних окремим записом користувача з роллю guest, що дає змогу зберігати для нього дані (наприклад, історію) ще до реєстрації.

4.4 Підтвердження електронної пошти

Під час реєстрації створюється непідтверджений обліковий запис і генерується одноразовий токен підтвердження, хеш якого зберігається у базі даних. Лист із посиланням надсилається асинхронно через чергу BullMQ поверх Redis [25], що не блокує відповідь сервера. Перехід за посиланням підтверджує пошту, споживає токен, видає сесійні cookie та перенаправляє користувача до застосунку, завдяки чому одразу після підтвердження він уже авторизований.

4.5 Завантаження та валідація карт

Адміністратор завантажує SVG-файли поверхів через захищені кінцеві точки. Розбір файлів виконується серверним DOM на основі бібліотеки jsdom відповідно до специфікації SVG [26]. З міркувань безпеки парсер відхиляє потенційно небезпечні конструкції: оголошення DOCTYPE, зовнішні сутності, елементи script і foreignObject, вбудовані обробники подій та небезпечні посилання.

Після успішного розбору формується граф поверху, виконується валідація рівня поверху та рівня будівлі (підрозділ 3.4), а вихідний файл зберігається в об’єктному сховищі. Метадані ресурсу карти (контрольна сума, кількість вузлів, ребер і кімнат, сам граф у форматі JSONB) записуються у базу даних з версіонуванням, що дає змогу зберігати історію версій карти та за потреби активувати попередню. Відомості про приміщення синхронізуються в окрему таблицю для пошуку після кожного завантаження чи активації версії.

Перевірка зв’язності будівлі реалізована обходом у ширину по об’єднаному графу всіх поверхів. Граф вважається коректним, якщо обхід із довільного вузла досягає всіх інших вузлів, тобто граф утворює одну зв’язну компоненту. Спрощений псевдокод перевірки наведено нижче.

```
function isConnected(graph):
    if graph.nodes is empty: return true
    start := any node of graph
    visited := { start }
    queue := [ start ]
    while queue is not empty:
        node := queue.dequeue()
        for next in graph.neighbors(node):
            if next not in visited:
                visited.add(next)
                queue.enqueue(next)
    return size(visited) == size(graph.nodes)
```

Рисунок 4.3 – Псевдокод перевірки зв’язності графа будівлі

4.6 Побудова найкоротшого шляху

Для пошуку маршруту між приміщеннями використано алгоритм A* — інформований алгоритм пошуку на графі, що поєднує фактичну вартість шляху від початку з евристичною оцінкою відстані до цілі [6], [12]. На кожному кроці алгоритм обирає вершину з найменшим значенням оцінювальної функції

$$f(n) = g(n) + h(n), \tag{4.1}$$

де g(n) — сумарна вага ребер від початкової вершини до вершини n, а h(n) — евристична оцінка вартості шляху від n до цілі. Як евристику використано евклідову відстань між координатами вузлів

$$h(n) = \sqrt{((x_n - x_g)^2 + (y_n - y_g)^2)}, \tag{4.2}$$

де (x_n, y_n) — координати поточного вузла, а (x_g, y_g) — координати цільового. Така евристика є допустимою (не переоцінює реальну відстань), тому алгоритм A* гарантовано знаходить найкоротший шлях [3]. Якщо координати вузла невідомі, евристика приймається рівною нулю, і алгоритм коректно вироджується у пошук Дейкстри.

Множину відкритих вершин реалізовано на основі двійкової купи (мінімальної піраміди) за ключем f(n), що забезпечує ефективне вилучення вершини з найменшою оцінкою [3]. Граф будівлі завантажується об’єднанням графів усіх опублікованих поверхів у єдину неорієнтовану структуру суміжності та кешується для повторних запитів. Початковий і кінцевий вузли визначаються за первинними вузлами навігації відповідних приміщень.

```
function A*(graph, start, goal):
    open  := min-heap keyed by f
    g[start] := 0
    f[start] := h(start, goal)
    open.push(start, f[start])
    while open is not empty:
        current := open.pop()      // вузол з мінімальним f
        if current == goal:
            return reconstruct(cameFrom, current)
        for (next, w) in graph.neighbors(current):
            tentative := g[current] + w
            if tentative < g[next]:
                cameFrom[next] := current
                g[next] := tentative
                f[next] := tentative + h(next, goal)
                open.push(next, f[next])
    return null                    // шлях не існує
```

Рисунок 4.4 – Псевдокод алгоритму A* пошуку найкоротшого шляху

Результатом є об’єкт маршруту, що містить коди початкового та кінцевого приміщень, ідентифікатори відповідних вузлів, сумарну довжину, кількість переходів, перелік пройдених поверхів та впорядковану послідовність кроків (для кожного — ідентифікатор вузла, номер поверху та вагу відрізка від попереднього вузла). Маршрути між поверхами будуються завдяки спільним міжповерховим вузлам сходів. Кожен успішний запит авторизованого користувача записується в історію маршрутів.

4.7 Пошук приміщень і категорії

Пошук приміщень виконується за назвою та ключовими словами у межах усього корпусу. Для кожного приміщення формується зведене пошукове поле (haystack), за яким побудовано індекс на основі триграм PostgreSQL [22]. Це забезпечує швидкий нечіткий пошук із підказками під час введення навіть за наявності друкарських помилок. Додатково підтримується фільтрування за категоріями приміщень, що дає змогу одночасно підсвічувати на плані всі кімнати обраних типів.

Запити пошуку обмежуються контекстом обраного корпусу, а результати впорядковуються за релевантністю, тож найточніші збіги показуються першими. На клієнті введення супроводжується затримкою (debounce), щоб не надсилати

запит на кожне натискання клавіші, а отримані варіанти відображаються у вигляді підказок. Вибір підказки фокусує карту на знайденому приміщенні та підсвічує його, що замикає сценарій пошуку у єдиний плавний робочий процес.

4.8 Історія пошуків і маршрутів

Система веде дві історії: історію пошукових запитів приміщень та історію побудованих маршрутів. Записи прив’язано до користувача, тож після входу він бачить власну історію та може повторно застосувати будь-який попередній пошук чи маршрут одним дотиком. Передбачено вилучення окремих записів та повне очищення історії.

Для гостей історія ведеться у межах їхнього тимчасового облікового запису, тому навіть неавторизований відвідувач отримує контекст попередніх дій у поточному сеансі. Після реєстрації гостьові дані можуть бути збережені за користувачем, що забезпечує плавний перехід від анонімного використання до повноцінного облікового запису без втрати накопиченої історії.

4.9 Збережені маршрути та приміщення

Зареєстрований користувач може зберігати приміщення та маршрути для швидкого повторного доступу. Збережені елементи відображаються в окремій панелі інтерфейсу, звідки їх можна відкрити на карті або видалити. Це зручно для регулярних маршрутів (наприклад, до власної кафедри чи улюбленої аудиторії).

Збережені елементи зберігаються у таблиці saved_items з посиланням на користувача та тип елемента (приміщення або маршрут), що дає змогу уніфіковано працювати з обома видами. Відкриття збереженого маршруту одразу відновлює його відображення на карті з потрібним поверхом, а відкриття збереженого приміщення фокусує карту на ньому, що економить час під час повторних звернень.

4.10 Події університету

Модуль подій надає інформацію про заходи університету, прив’язані до конкретних приміщень і часу проведення. Користувач переглядає перелік подій, а

вибравши подію, бачить її розташування на карті та може одразу прокласти до неї маршрут. Адміністратор створює та редагує події через інтерфейс адміністрування.

Кожна подія зберігає назву, опис, час початку й завершення та посилання на приміщення, у якому вона відбувається. Завдяки цьому зв’язку модуль подій повторно використовує наявну навігаційну інфраструктуру: побудова маршруту до події нічим не відрізняється від побудови маршруту до приміщення. Перелік подій можна фільтрувати за датою, що допомагає користувачеві планувати відвідування заходів.

4.11 Адміністрування

Інтерфейс адміністратора, доступний лише для ролі admin, забезпечує повний життєвий цикл картографічних даних: завантаження SVG-поверхів, перегляд версій і активацію потрібної версії, фіналізацію корпусу з перевіркою зв’язності, редагування відомостей про корпуси та приміщення, а також керування подіями. Усі адміністративні операції захищено стражем ролей.

Процес введення нового корпусу в експлуатацію має чітко визначений життєвий цикл. Спершу адміністратор створює корпус у стані чернетки та завантажує карти його поверхів; на цьому етапі дані видимі лише адміністраторові. Після завантаження всіх поверхів виконується фіналізація з перевіркою глобальної зв’язності, і лише в разі успіху корпус переходить у стан «опублікований» та стає доступним користувачам. Такий контрольований перехід унеможливлює публікацію неповних або некоректних карт і робить процес адміністрування передбачуваним.

4.12 Клієнтська частина

Клієнтський застосунок побудовано на Next.js і React [20], [23]. Карта поверху завантажується як SVG через власний проксі-маршрут Next.js, що звертається до сервера зі збереженням cookie та обходить обмеження CORS для підписаних посилань сховища. Отриманий SVG проходить санітизацію (видалення сценаріїв і обробників подій) і вбудовується у DOM, після чого до нього додаються обробники подій, класи підсвічування та накладання маршруту.

Масштабування й переміщення карти реалізовано бібліотекою react-zoom-pan-pinch. Маршрут відображається поверх плану шляхом розкриття прихованих шарів реальних ребер і вузлів графа, що відповідають крокам знайденого шляху, тож лінія маршруту точно повторює геометрію коридорів. Підсвічування знайдених приміщень виконується червоним контуром.

Стан клієнта поділено між React Query (серверні дані з кешуванням) і Zustand (локальний стан інтерфейсу) [24], [30]. Двомовність (українська та англійська) забезпечує бібліотека next-intl з маршрутизацією за локаллю. HTTP-клієнт axios має перехоплювач відповідей, який у разі відповіді 401 автоматично оновлює токен доступу та повторює запит, серіалізуючи одночасні спроби оновлення за допомогою м’ютекса. Візуальну мову інтерфейсу реалізовано за дизайн-системою Material Design 3 поверх Tailwind CSS [18], [27].

4.13 Тестування та перевірка працездатності

Правильність роботи системи підтверджено автоматизованими тестами та ручним випробуванням. Модульні тести перевіряють окремі одиниці бізнес-логіки — сервіси автентифікації, валідації карт, пошуку шляхів та пошуку приміщень, — а також контролери. Загалом написано 84 модульні тести, усі вони успішно проходять (рисунок наведено нижче).

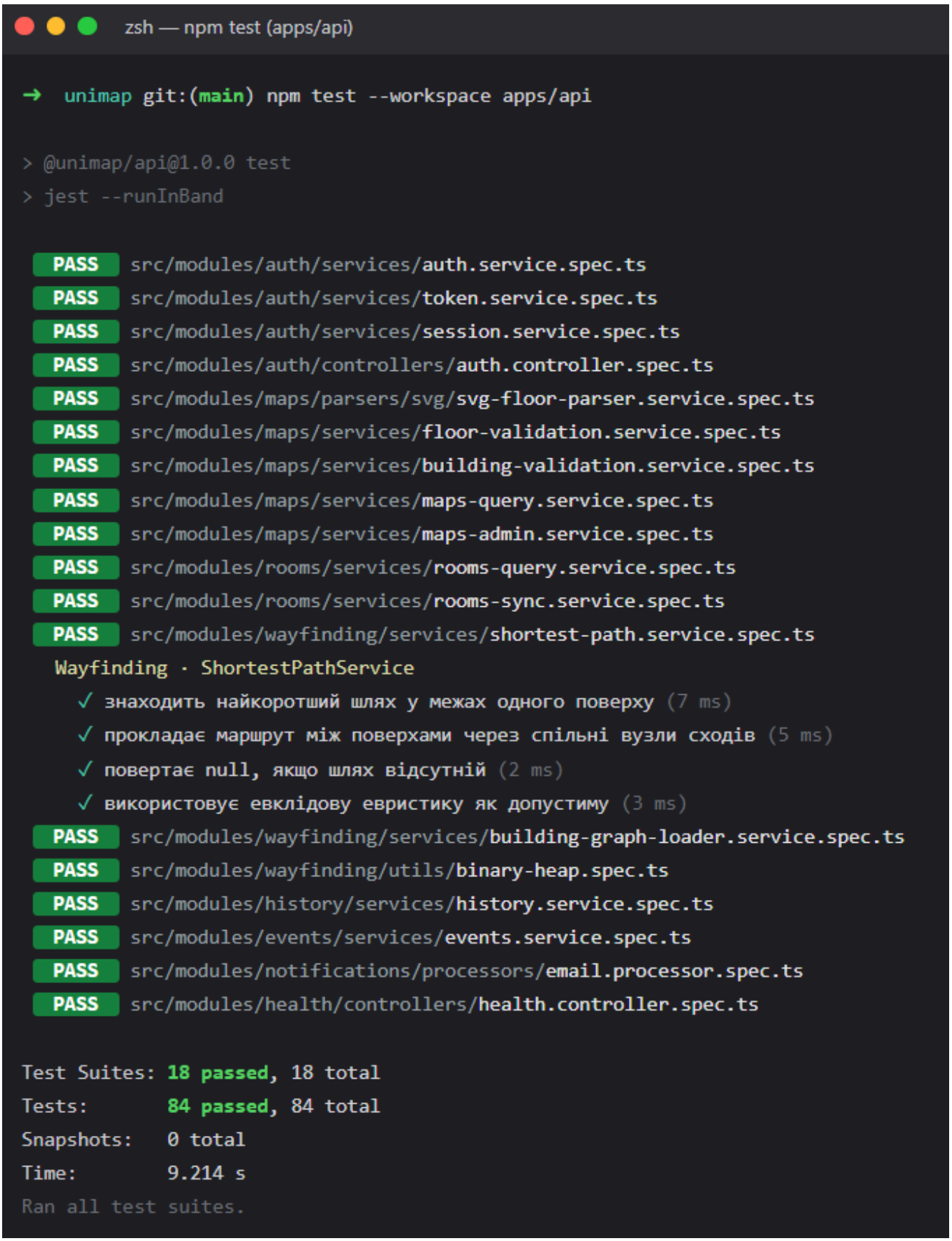


Рисунок 4.5 – Результат виконання модульних тестів

Інтеграційні тести перевіряють спільну роботу модулів: повний сценарій реєстрації та входу, завантаження карти з перевіркою зв’язності, побудову маршруту між поверхами та роботу з історією. Усі інтеграційні тести також проходять успішно.

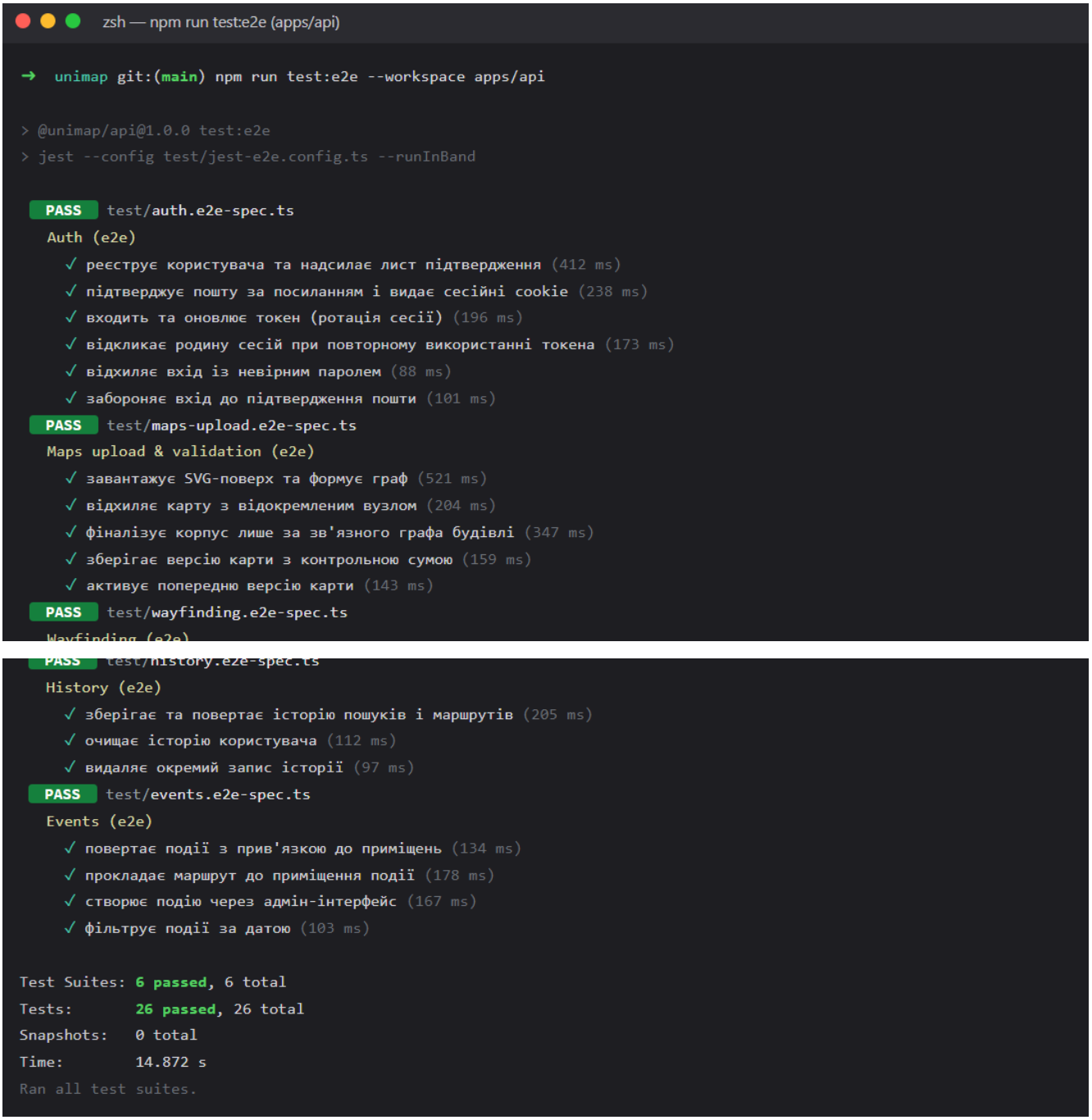


Рисунок 4.6 – Результат виконання інтеграційних тестів

Додатково проведено ручне тестування основних користувацьких сценаріїв у різних браузерах і на пристроях з різною шириною екрана, що підтвердило

коректність відображення, адаптивність інтерфейсу та відповідність функціональним вимогам, сформульованим у підрозділі 1.3.

4.14 Обробка помилок, журналювання та конфігурація

Для передбачуваної поведінки системи реалізовано наскрізну обробку помилок. Доменні помилки (наприклад, відсутність корпусу, недоступність маршруту, некоректні облікові дані) представлено окремими класами винятків, які глобальний фільтр винятків перетворює на узгоджені HTTP-відповіді з відповідним кодом стану та зрозумілим повідомленням. Це відокремлює бізнес-логіку від деталей транспортного рівня й робить відповіді інтерфейсу однорідними.

Усі запити проходять через перехоплювач журналювання, що фіксує метод, шлях, час обробки та результат. Журнали полегшують діагностику й моніторинг роботи системи в експлуатації. Конфігурація застосунку повністю винесена у змінні середовища та перевіряється схемами Zod на старті, тож некоректне налаштування виявляється негайно, а не під час обробки запитів [29].

4.15 Продуктивність та кешування

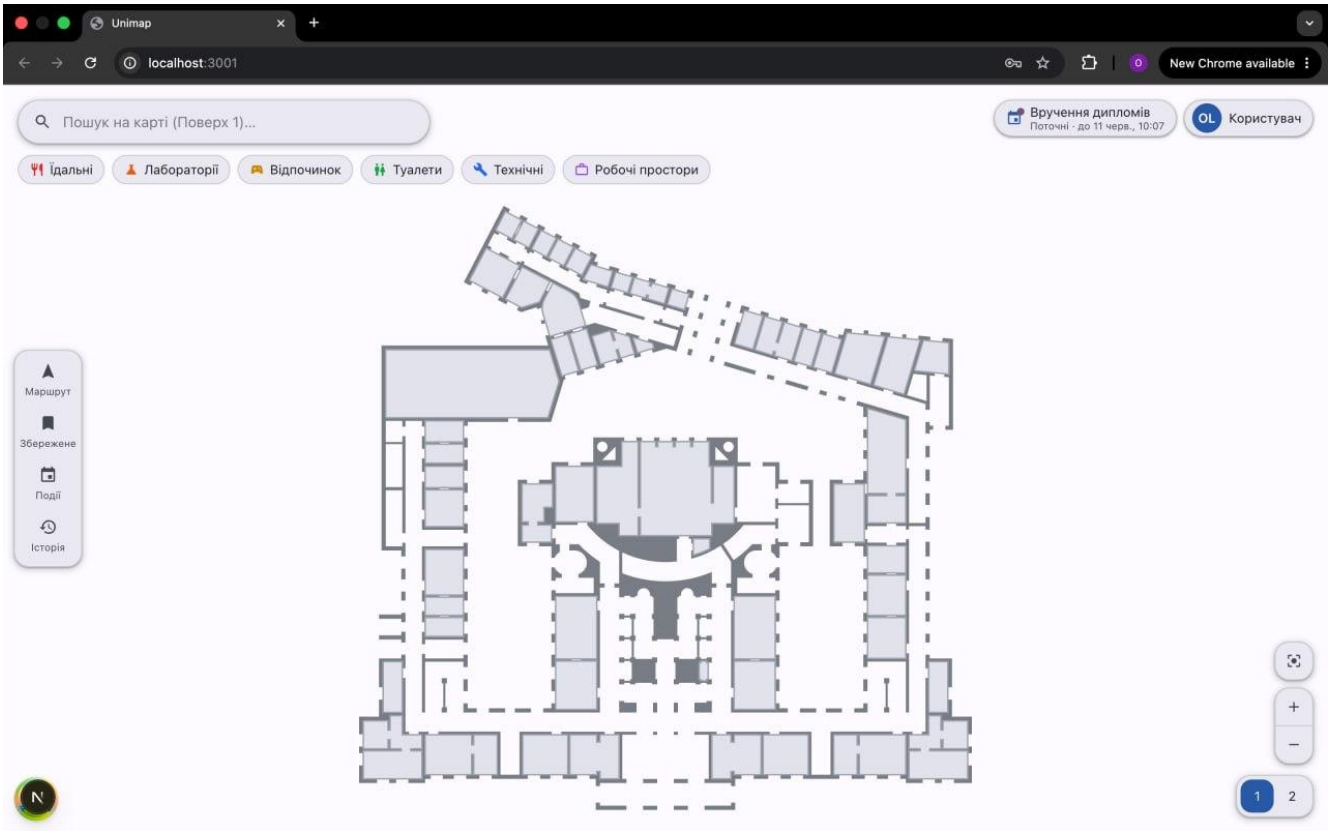
Для зменшення затримок застосовано кешування на кількох рівнях. На сервері графи опублікованих будівель завантажуються один раз і зберігаються у пам’яті, тому повторні запити маршрутів не потребують повторного злиття графів поверхів. Часті запити до даних, що змінюються рідко (перелік корпусів, категорії), також кешуються. На клієнті бібліотека React Query кешує відповіді сервера й уникає зайвих повторних запитів, а великі статичні SVG-файли віддаються безпосередньо з об’єктного сховища через підписані посилання [24], [7]. Сукупно ці заходи забезпечують швидку побудову маршрутів і плавну взаємодію з картою навіть на пристроях середньої потужності.

РОЗДІЛ 5
ОПИС ІНТЕРФЕЙСУ КОРИСТУВАЧА

Інтерфейс застосунку спроектовано за принципами зручності використання та інтуїтивної зрозумілості [8], [10] і за візуальною мовою Material Design 3 [18]. Композицію елементів керування навмисно наближено до звичних користувачам картографічних сервісів на кшталт Google Maps, щоб скоротити час навчання. У цьому розділі покроково описано основні екрани та сценарії роботи.

5.1 Головна сторінка та інтерактивна карта

Основну частину екрана займає інтерактивна карта поверху. Зверху розташовано рядок пошуку у вигляді «пігулки» та горизонтальний перелік категорій. Праворуч згруповано елементи керування масштабом, ліворуч (або внизу на мобільних екранах) — панель інструментів. Карту можна переміщувати перетягуванням і масштабувати.



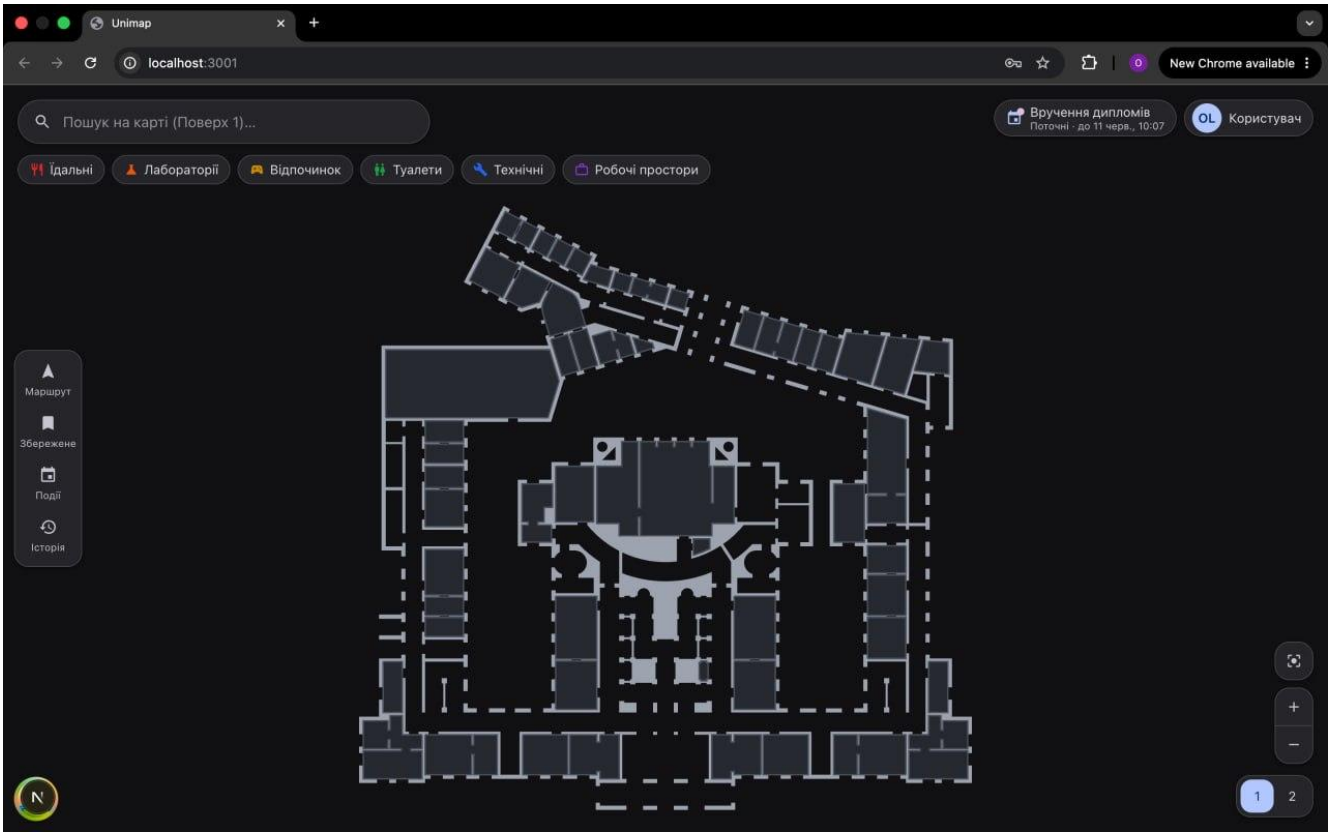


Рисунок 5.1 – Головна сторінка застосунку з інтерактивною картою

Композицію побудовано так, щоб карта залишалася в центрі уваги, а елементи керування не перекривали корисний простір. Прозорі для сприйняття, але водночас помітні панелі розміщено по краях екрана, що відповідає принципу мінімізації візуального шуму [10]. Меню користувача у правому верхньому куті надає доступ до входу, налаштувань мови й теми та персональних розділів.

5.2 Пошук приміщень

Під час введення запиту в рядку пошуку з’являються підказки з відповідними приміщеннями всього корпусу. Після вибору приміщення карта плавно зміщує фокус і масштаб на знайдену кімнату та підсвічує її червоним контуром.

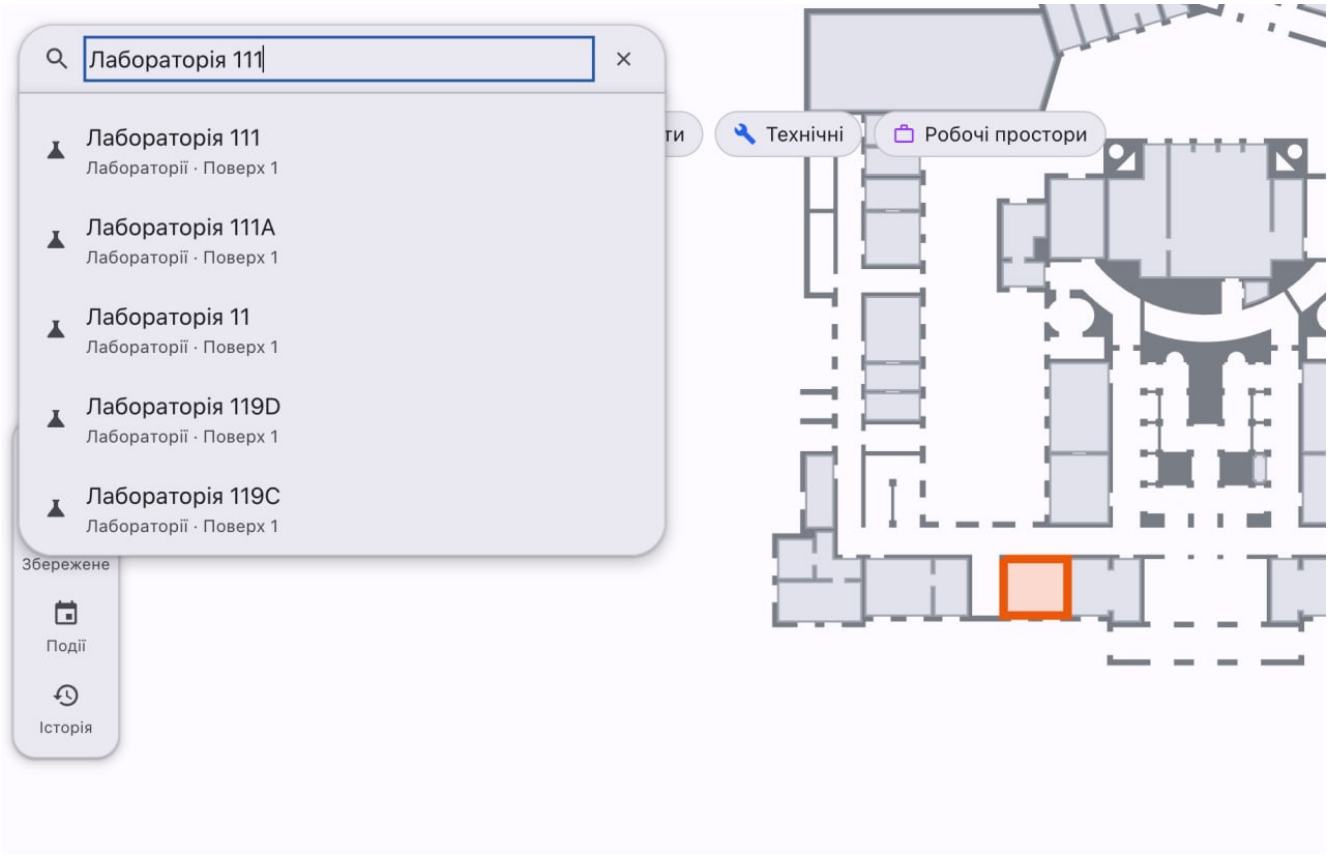


Рисунок 5.2 – Пошук приміщення за назвою

5.3 Фільтрування за категоріями

Кнопки категорій працюють як прапорці множинного вибору і відображаються різними кольорами. Після вибору однієї або кількох категорій на плані автоматично підсвічуються всі приміщення відповідних типів, що спрощує пошук, наприклад, найближчого санвузла або їдальні.

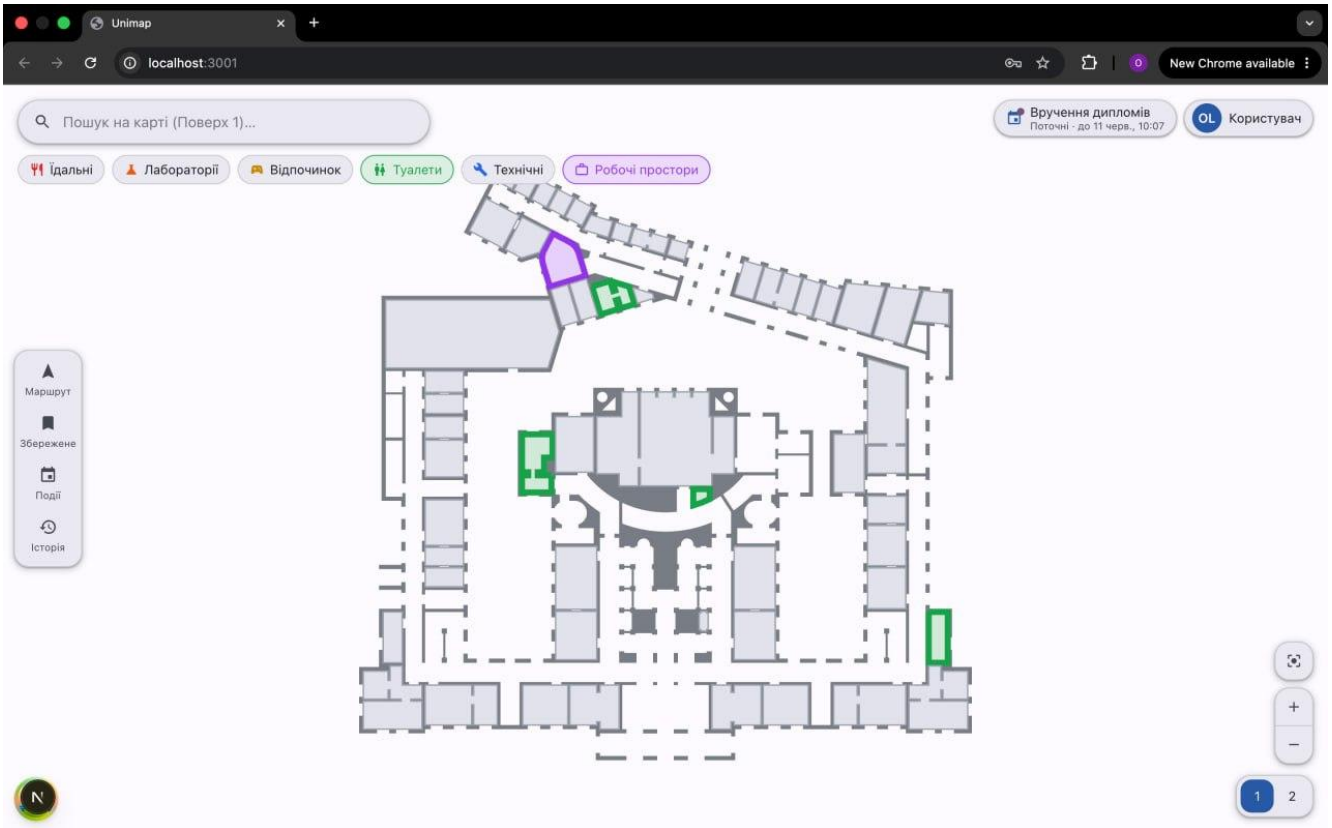


Рисунок 5.3 – Фільтрування приміщень за категоріями

5.4 Деталі приміщення

Дотик до приміщення на карті відкриває модальне вікно з його назвою, категорією, поверхом та іншими відомостями. Звідси можна одразу перейти до побудувати маршруту або додати до збережених.

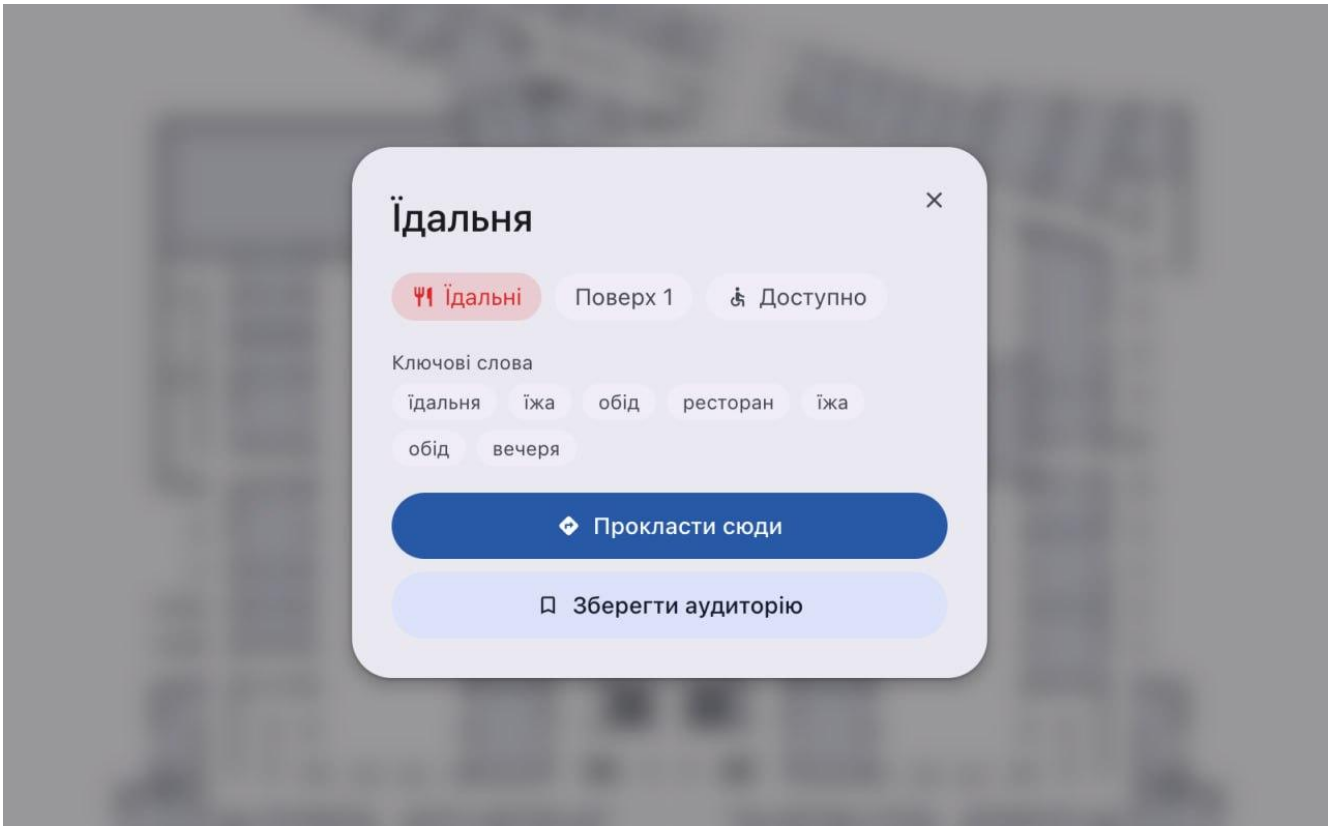
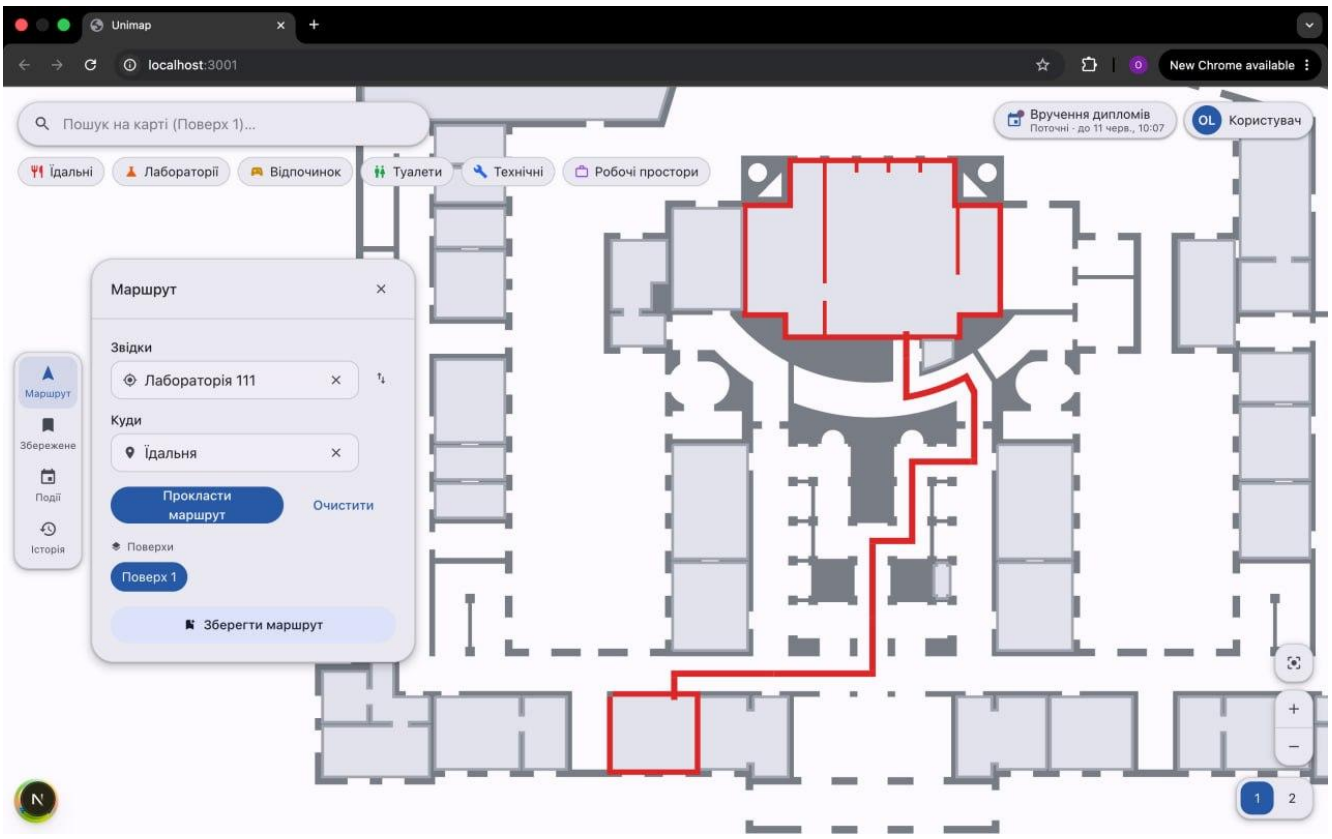
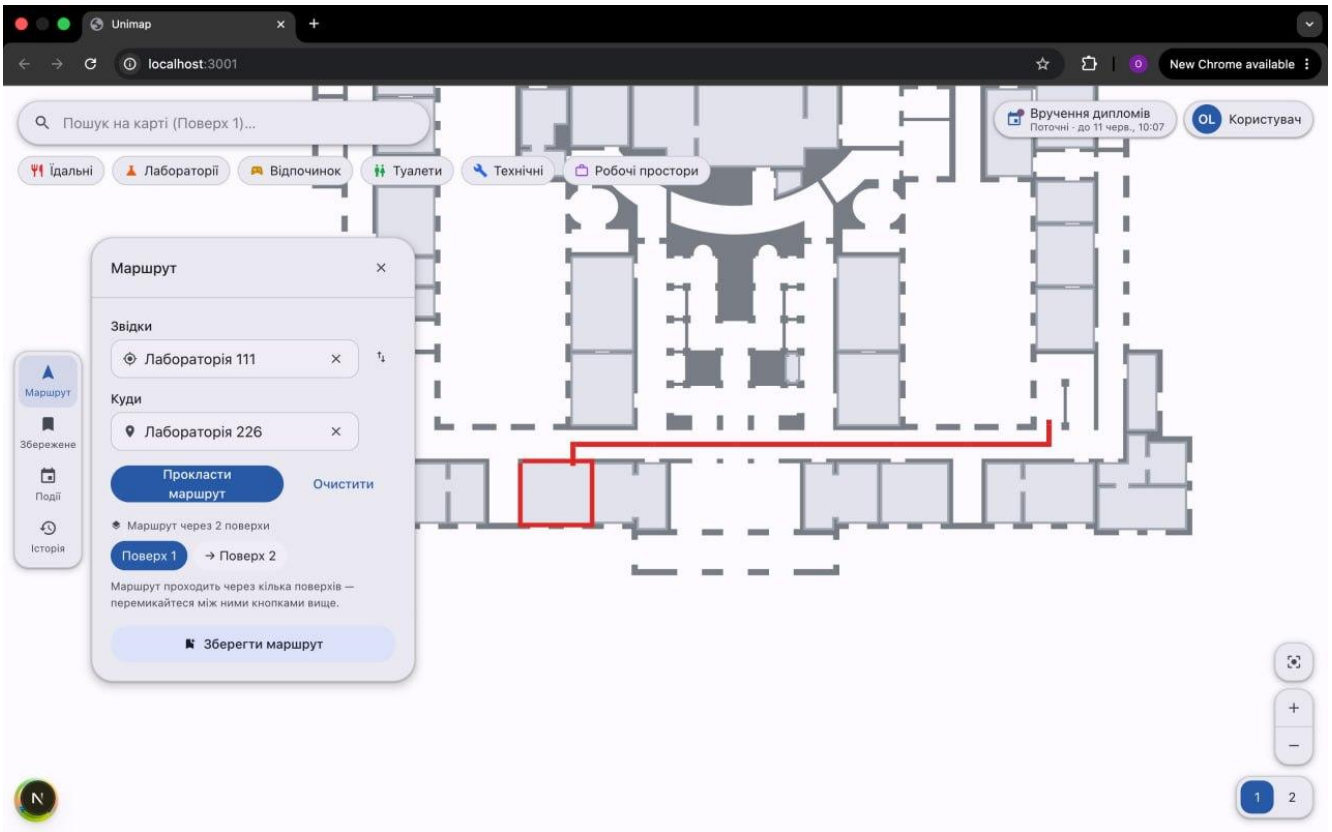


Рисунок 5.4 – Модальне вікно з детальною інформацією про приміщення

5.5 Навігація та побудова маршруту

Інструмент навігації відкриває форму з полями початкового та кінцевого приміщень. Після побудови маршрут відображається поверх плану лінією, що повторює геометрію коридорів, з позначенням початкової та кінцевої точок. Для маршрутів між поверхами передбачено перемикання поверхів із збереженням відповідного фрагмента маршруту.


Unimap
localhost:3001
New Chrome available
Пошук на карті (Поверх 1)...
Вручення дипломів
Поточні - до 11 черв., 10:07
OL
Користувач
Їдальні
Лабораторії
Відпочинок
Туалети
Технічні
Робочі простори
Маршрут
Звідки
Лабораторія 111
Куди
Їдальня
Прокласти маршрут
Очистити
Поверхи
Поверх 1
Зберегти маршрут
Маршрут
Збережене
Події
Історія
1
2


Unimap
localhost:3001
New Chrome available
Пошук на карті (Поверх 1)...
Вручення дипломів
Поточні - до 11 черв., 10:07
OL
Користувач
Їдальні
Лабораторії
Відпочинок
Туалети
Технічні
Робочі простори
Маршрут
Звідки
Лабораторія 111
Куди
Лабораторія 226
Прокласти маршрут
Очистити
Маршрут через 2 поверхи
Поверх 1
→ Поверх 2
Маршрут проходить через кілька поверхів — перемикайтеся між ними кнопками вище.
Зберегти маршрут
Маршрут
Збережене
Події
Історія
1
2

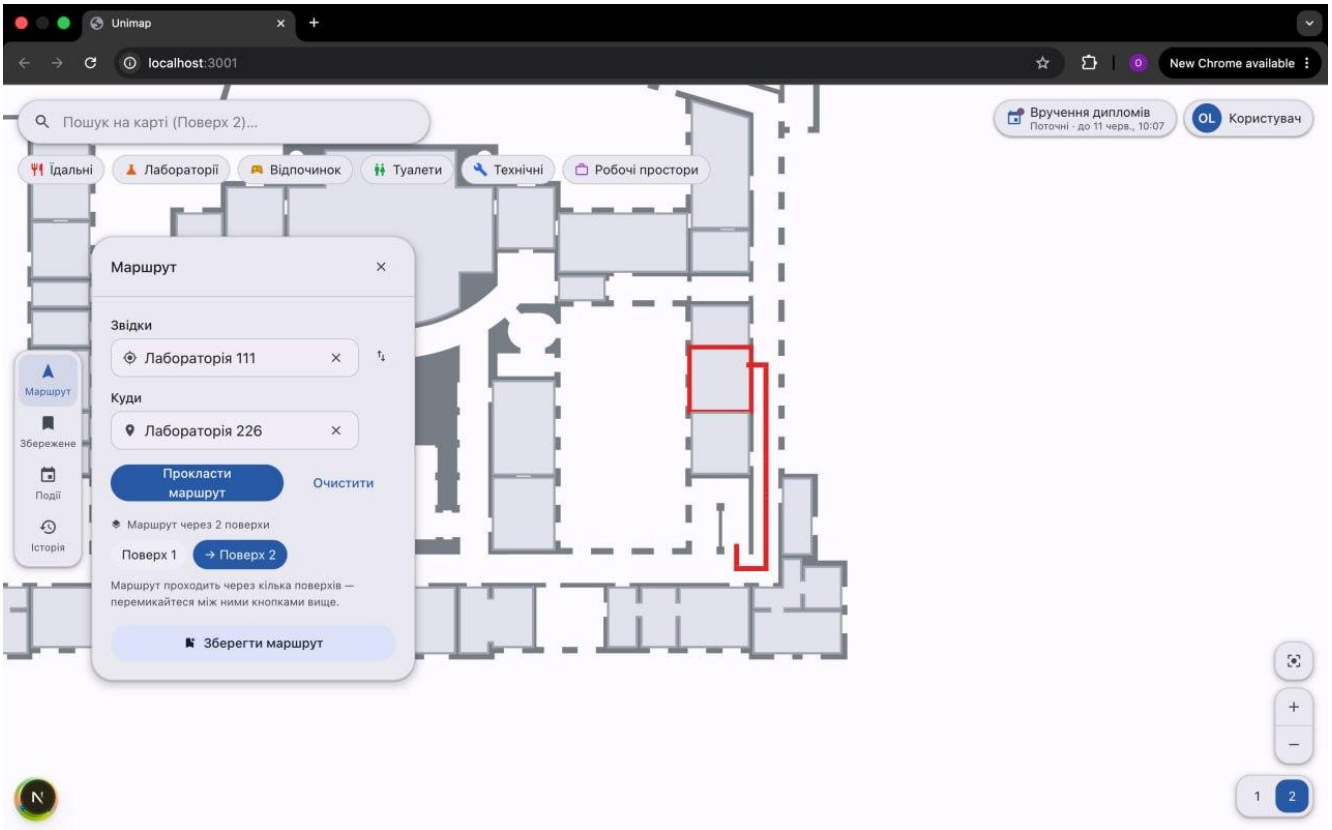


Рисунок 5.6 – Побудовані маршрути між приміщеннями поверх плану

Поля початкового та кінцевого приміщень підтримують той самий пошук із підказками, що й основний рядок пошуку, тому користувач може швидко задати обидві точки маршруту. Якщо одну з точок уже вибрано на карті (наприклад, через картку приміщення), відповідне поле заповнюється автоматично. Побудований маршрут супроводжується відомостями про сумарну відстань і перелік поверхів, які він перетинає, що допомагає користувачеві оцінити шлях наперед.

5.6 Історія

Панель історії містить попередні пошукові запити та побудовані маршрути користувача. Будь-який запис можна повторно застосувати одним дотиком або видалити. Неавторизованому користувачеві у цій панелі пропонується ввійти, щоб переглянути збережену історію.

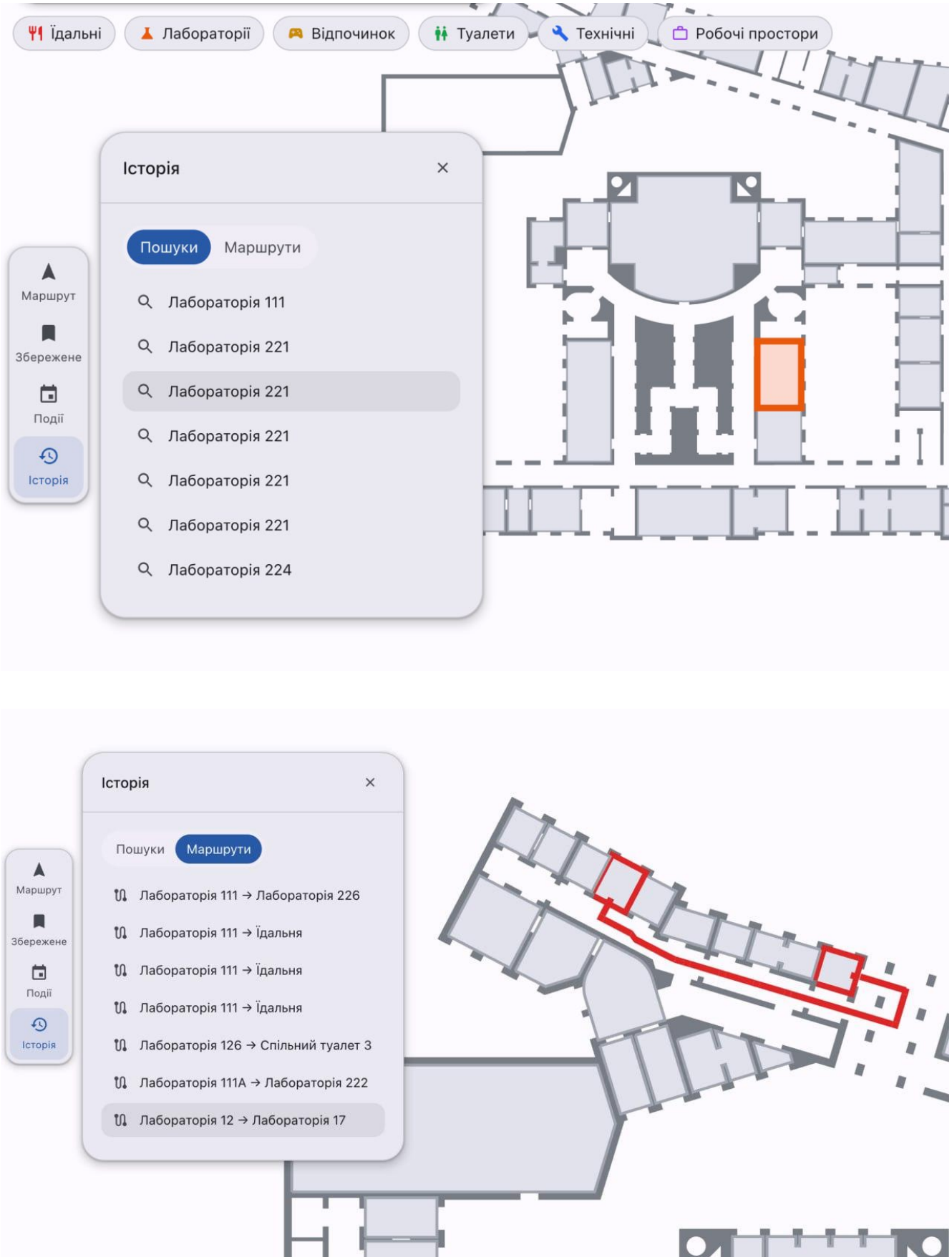


Рисунок 5.7 – Панель історії пошуків і маршрутів

5.7 Збережені приміщення та маршрути

Панель збереженого надає швидкий доступ до позначених користувачем приміщень і маршрутів. Елементи можна відкрити на карті або видалити зі списку.



Рисунок 5.8 – Панель збережених приміщень і маршрутів

5.8 Події університету

Розділ подій відображає заплановані заходи з прив’язкою до приміщень і часу. Вибравши подію, користувач бачить її розташування на карті та може прокласти до неї маршрут.

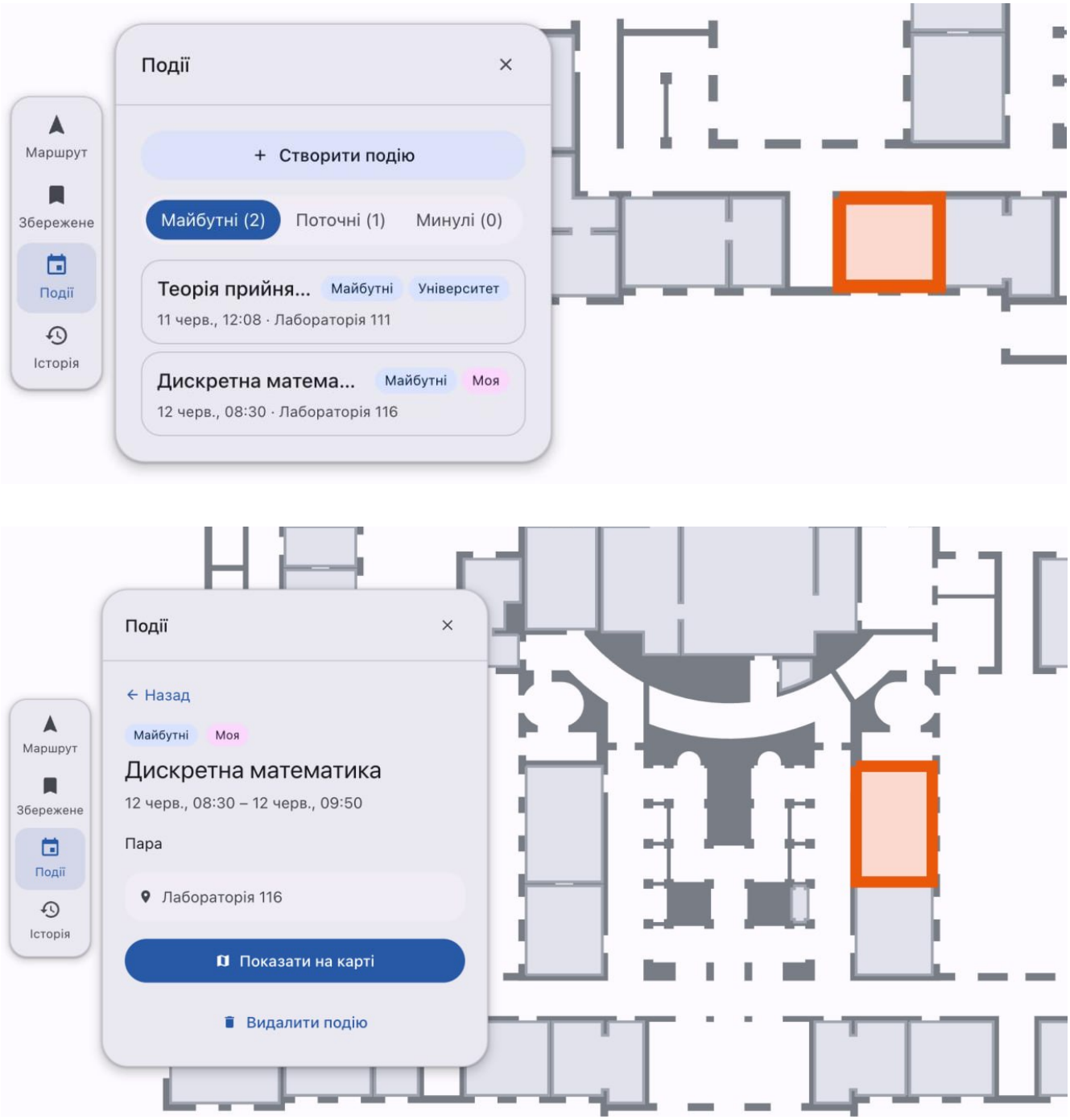


Рисунок 5.9 – Перелік подій університету з прив’язкою до приміщень

5.9 Вхід та реєстрація

Вхід і реєстрація виконуються у модальному вікні з двома вкладками. Форми побудовано на react-hook-form із валідацією схемами Zod [29], тож користувач одразу бачить підказки щодо помилок введення. Після реєстрації надсилається лист підтвердження, а перехід за посиланням автоматично авторизує користувача.

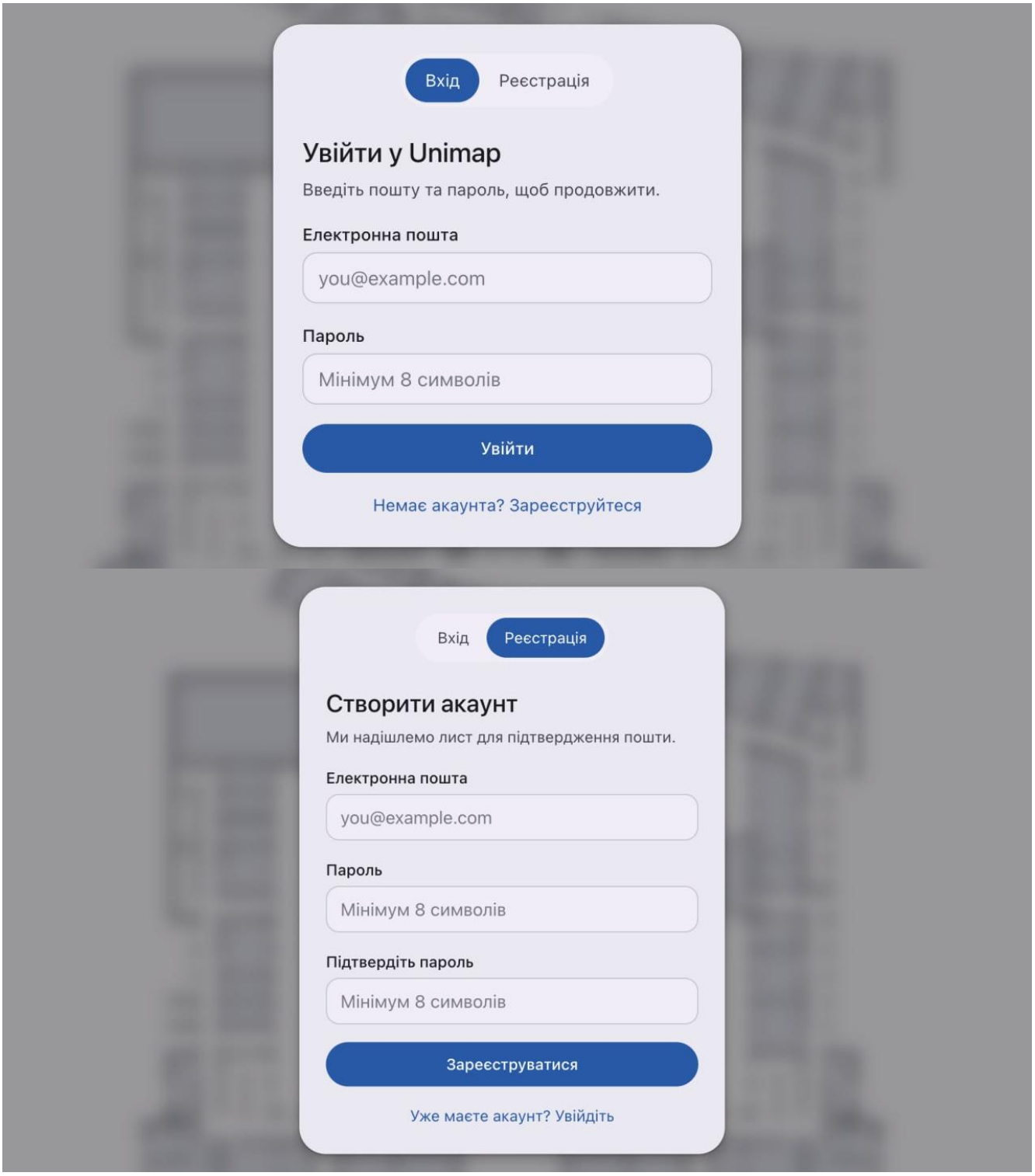
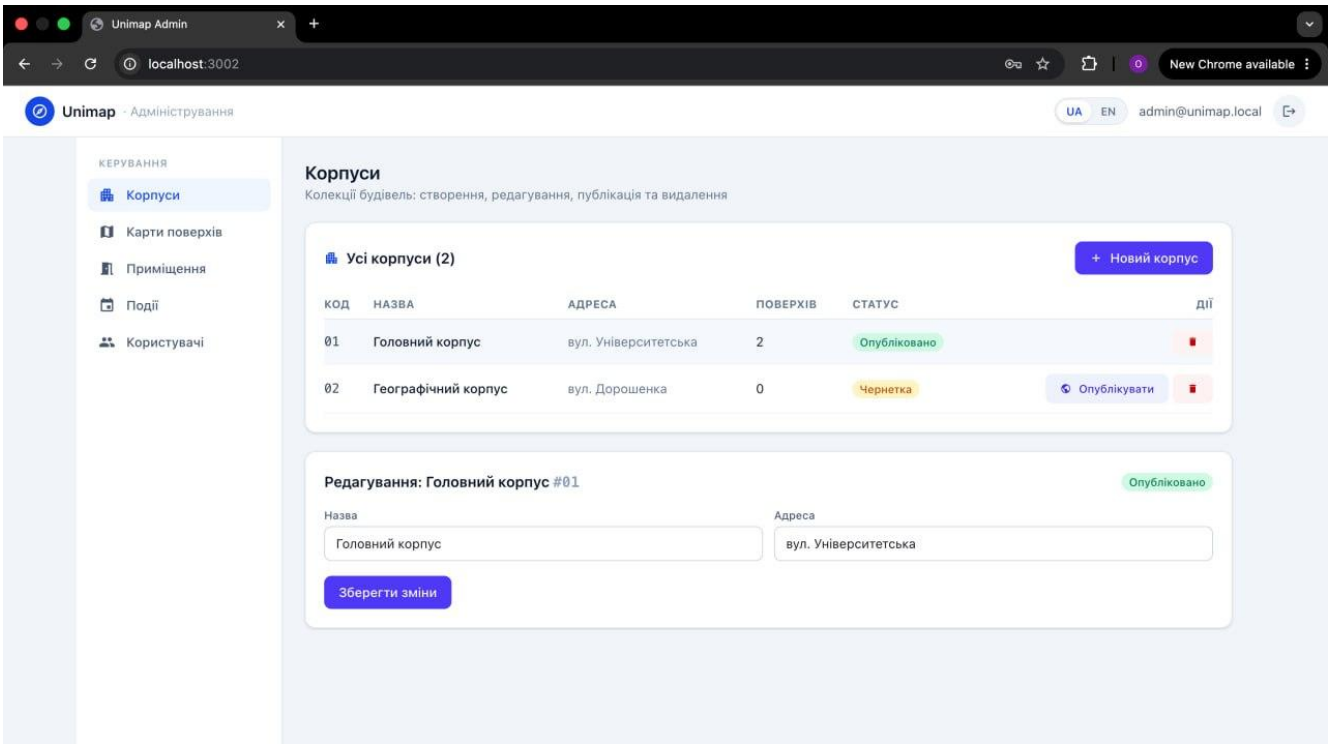
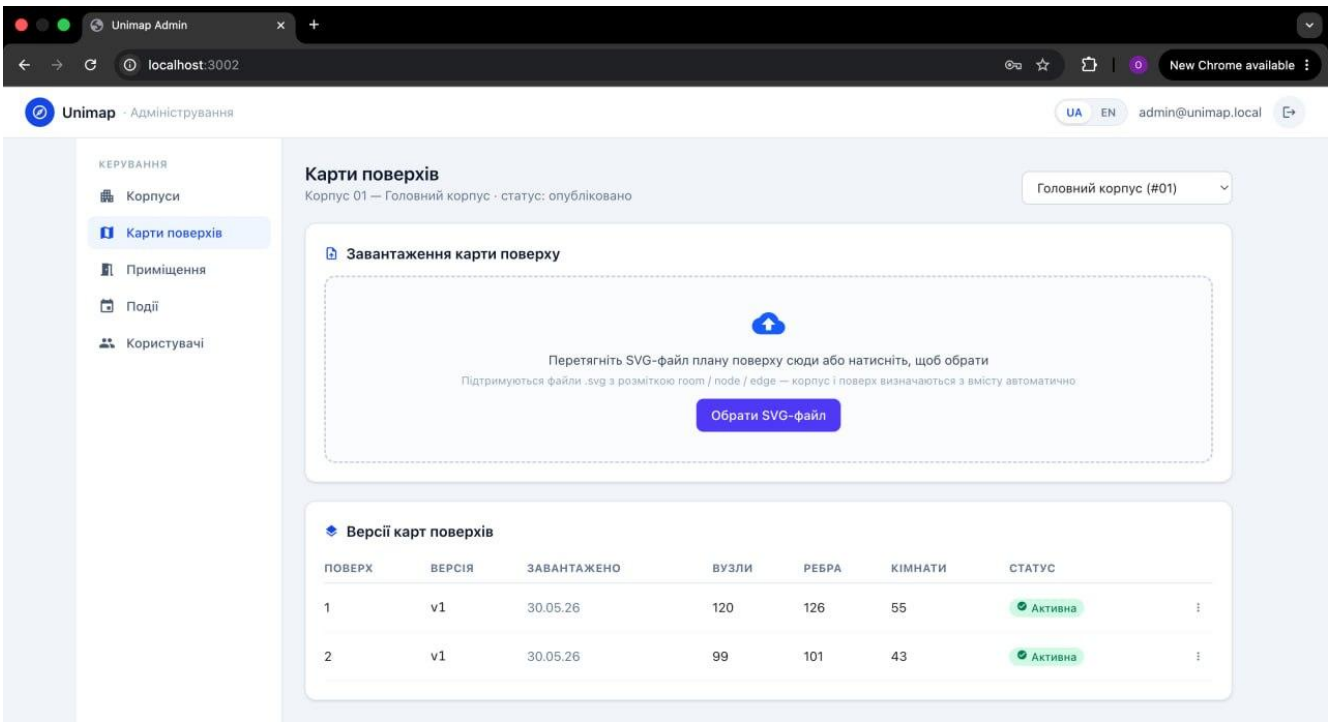


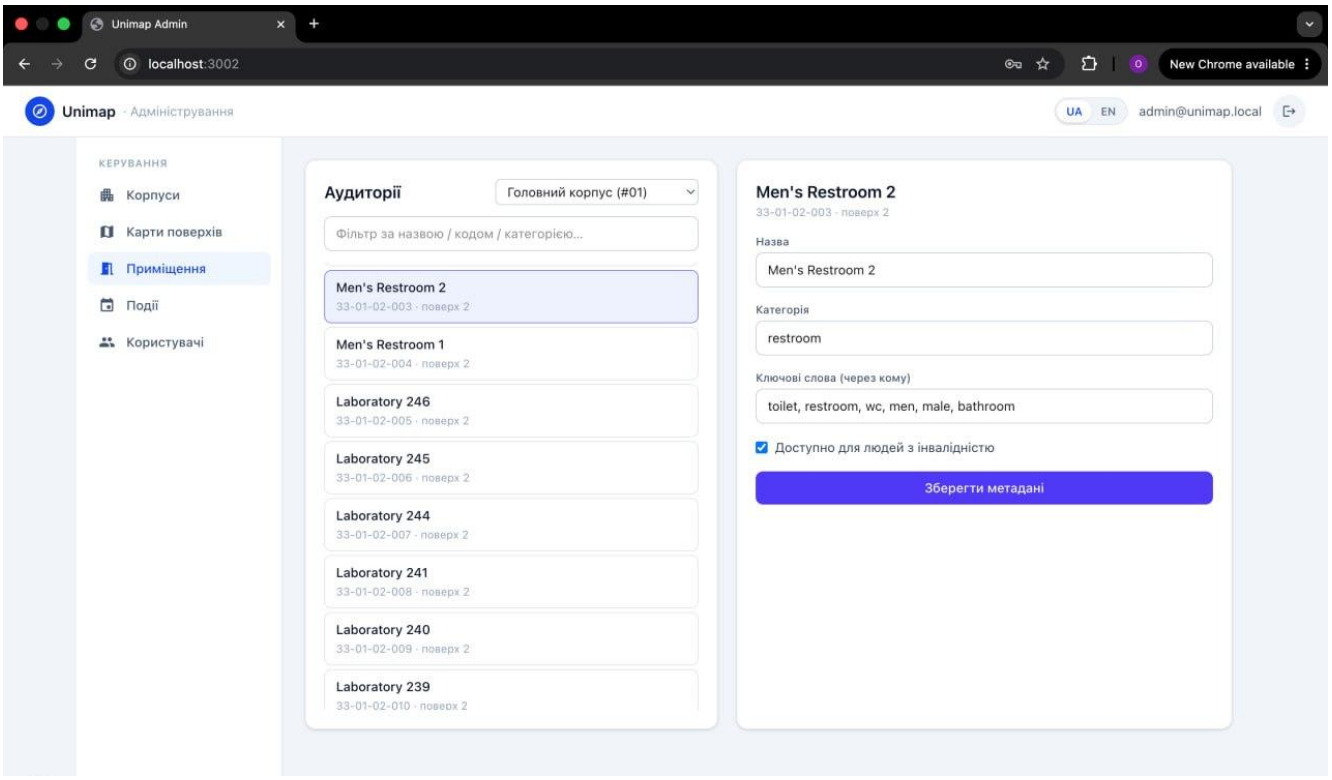
Рисунок 5.10 – Модальне вікно входу та реєстрації

5.10 Інтерфейс адміністратора

Адміністратор має доступ до окремого інтерфейсу керування картографічними даними: завантаження SVG-поверхів, перегляд версій карт і їх активація, фіналізація корпусу з перевіркою зв’язності, редагування корпусів і приміщень та керування подіями.




Unimap Admin
localhost:3002
New Chrome available
Unimap · Адміністрування
UA EN
admin@unimap.local
КЕРУВАННЯ
Корпуси
Карти поверхів
Приміщення
Події
Користувачі
Карти поверхів
Корпус 01 — Головний корпус · статус: опубліковано
Головний корпус (#01)
Завантаження карти поверху
Перетягніть SVG-файл плану поверху сюди або натисніть, щоб обрати
Підтримуються файли .svg з розміткою room / node / edge — корпус і поверх визначаються з вмісту автоматично
Обрати SVG-файл
Версії карт поверхів
ПОВЕРХ ВЕРСІЯ ЗАВАНТАЖЕНО ВУЗЛИ РЕБРА КІМНАТИ СТАТУС
1 v1 30.05.26 120 126 55 Активна
2 v1 30.05.26 99 101 43 Активна


Unimap Admin
localhost:3002
New Chrome available
Unimap · Адміністрування
UA EN
admin@unimap.local
КЕРУВАННЯ
Корпуси
Карти поверхів
Приміщення
Події
Користувачі
Аудиторії
Головний корпус (#01)
Фільтр за назвою / кодом / категорією...
Men's Restroom 2
33-01-02-003 · поверх 2
Men's Restroom 1
33-01-02-004 · поверх 2
Laboratory 246
33-01-02-005 · поверх 2
Laboratory 245
33-01-02-006 · поверх 2
Laboratory 244
33-01-02-007 · поверх 2
Laboratory 241
33-01-02-008 · поверх 2
Laboratory 240
33-01-02-009 · поверх 2
Laboratory 239
33-01-02-010 · поверх 2
Men's Restroom 2
33-01-02-003 · поверх 2
Назва
Men's Restroom 2
Категорія
restroom
Ключові слова (через кому)
toilet, restroom, wc, men, male, bathroom
Доступно для людей з інвалідністю
Зберегти метадані

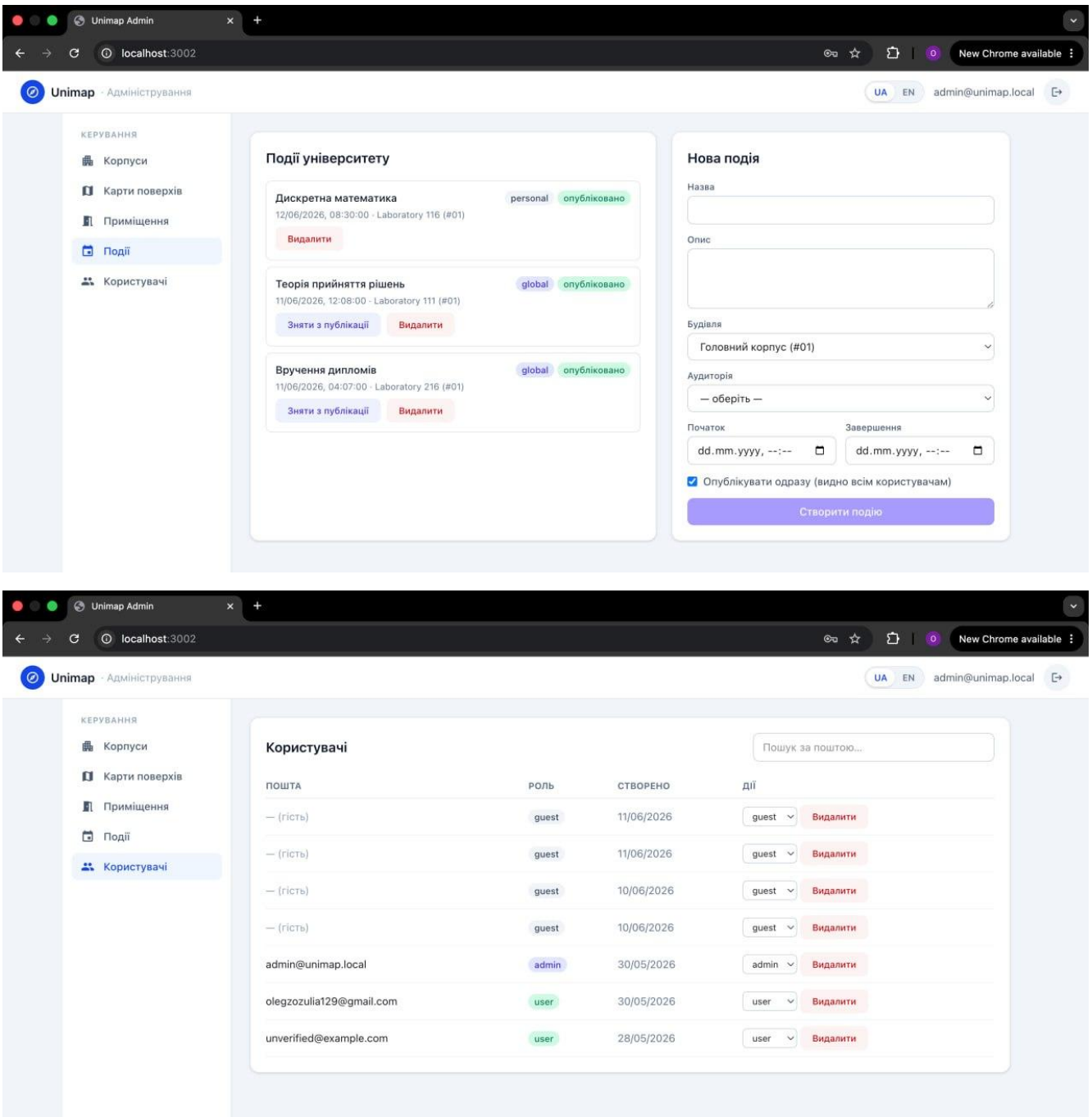


Рисунок 5.11 – Інтерфейс адмін-панелі

Інтерфейс адміністратора проводить його крок за кроком: від створення корпусу та завантаження поверхів до фіналізації з перевіркою зв’язності. У разі виявлення проблем (наприклад, відокремленого вузла) система виводить зрозуміле повідомлення із зазначенням місця помилки, що дає змогу швидко виправити вихідну карту. Завдяки цьому відповідальна, але потенційно складна операція підготовки даних залишається керованою навіть для нетехнічного персоналу.

5.11 Перемикання мови, теми та поверху

Меню користувача дає змогу перемикати мову інтерфейсу (українська та англійська) і світлу та темну теми, узгоджені з токенами Material Design 3 [18]. Перемикач поверхів розташовано на карті та забезпечує швидку зміну активного поверху корпусу зі збереженням контексту перегляду.

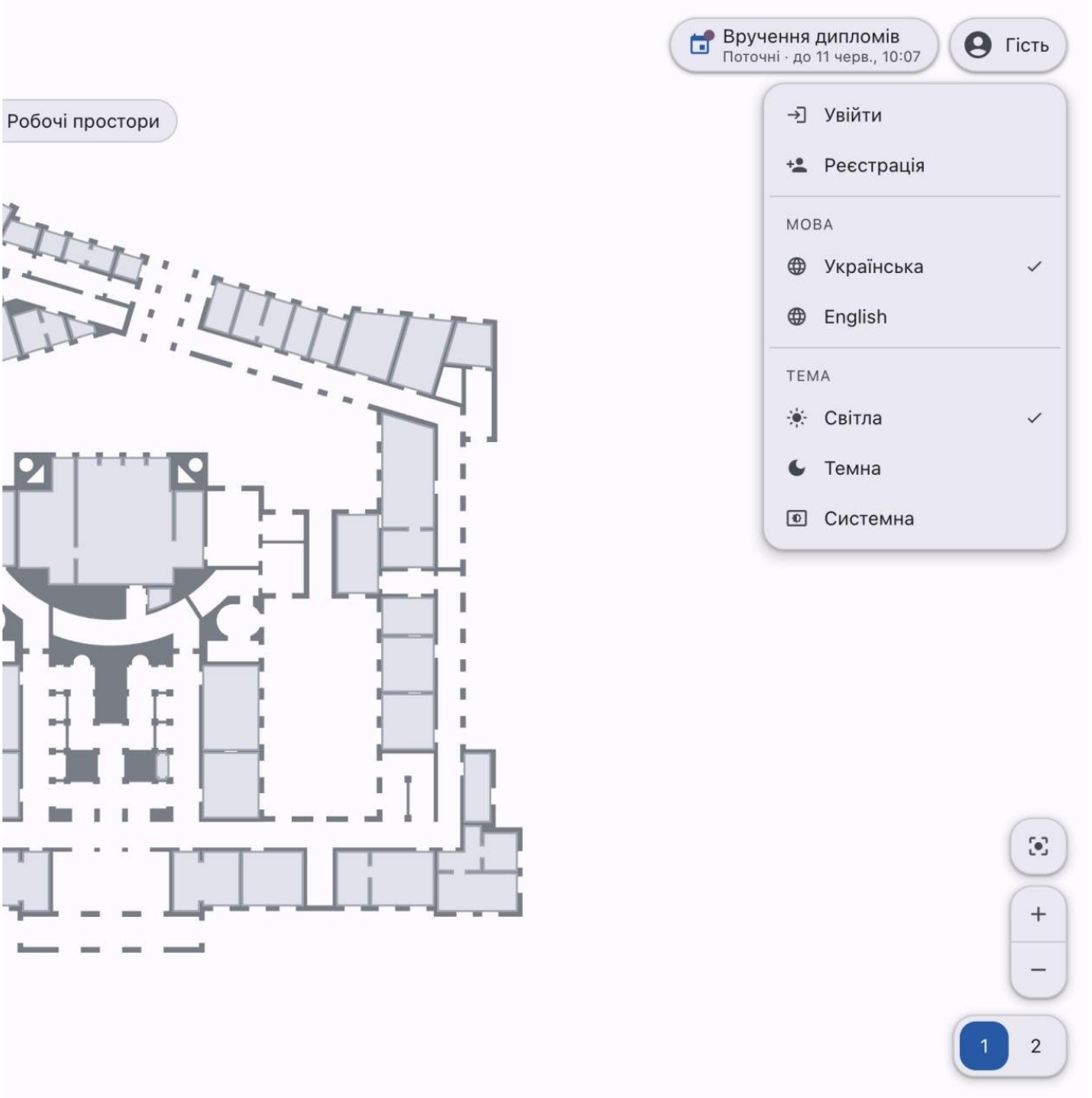


Рисунок 5.12 – Перемикання мови, теми та поверху будівлі

5.12 Адаптивність та мобільний вигляд

Інтерфейс адаптується до ширини екрана. На настільних пристроях панель інструментів розташована збоку, а на мобільних — згортається у нижню панель, що відповідає очікуванням користувачів мобільних застосунків [8]. Елементи керування мають достатній розмір для дотику, а рядок пошуку й перемикач поверхів залишаються доступними за будь-якої ширини екрана. Усі інтерактивні елементи (кнопки, категорії, приміщення на карті) мають візуальний відгук під час наведення та натискання, що покращує сприйняття інтерфейсу.

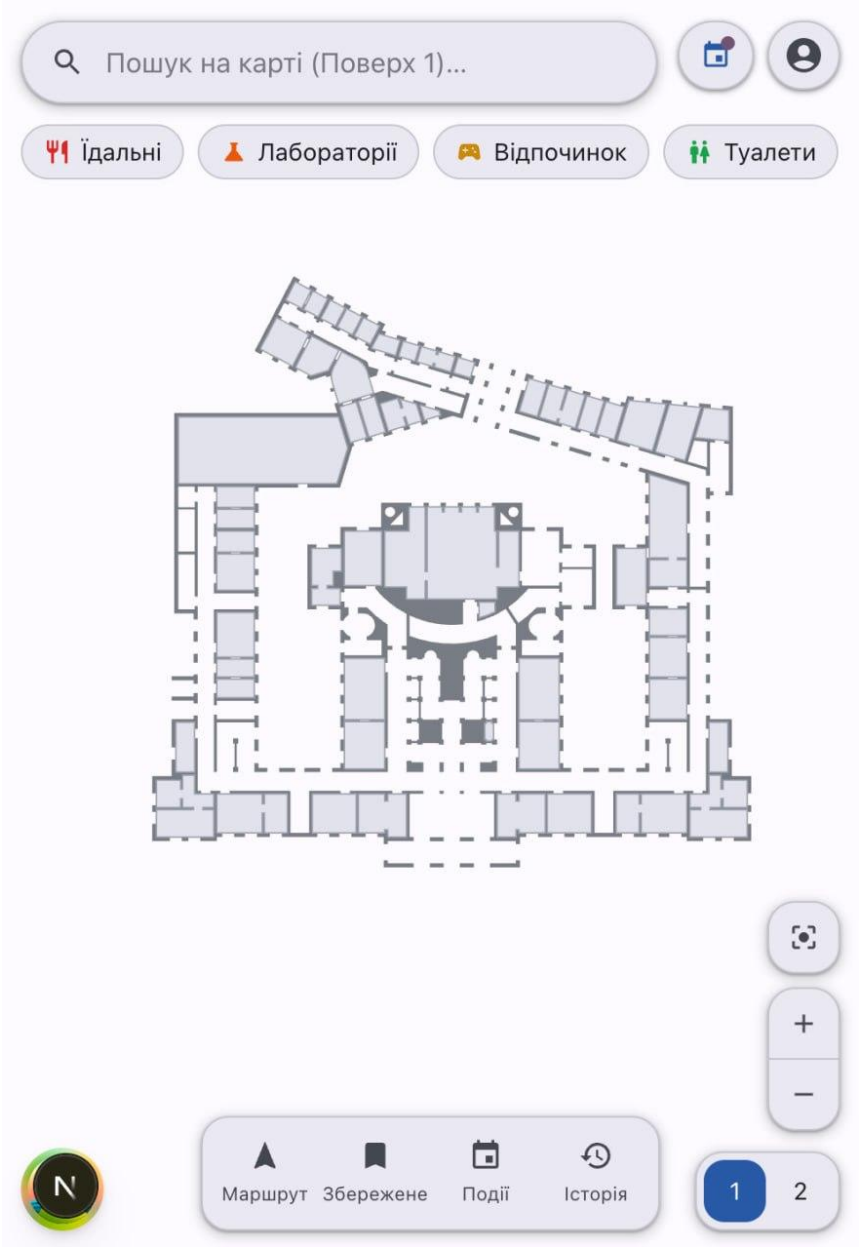


Рисунок 5.13 – Вигляд застосунку на мобільному пристрої

ВИСНОВКИ

У межах кваліфікаційної роботи спроектовано та розроблено веб-застосунок Unimap для інтерактивної навігації приміщеннями університету. Поставлену мету досягнуто, а всі визначені у вступі завдання виконано повністю.

Основні результати роботи такі:

1) проаналізовано предметну область і наявні рішення (статичні плани, Google Maps, спеціалізовані платформи), на підставі чого сформульовано функціональні та нефункціональні вимоги й обґрунтовано доцільність розробки власного продукту;
2) запропоновано та реалізовано модель плану поверху як зв’язного неорієнтованого графа на основі структурованого SVG з автоматичною перевіркою зв’язності на рівні поверху та будівлі, що гарантує коректність побудованих маршрутів;
3) розроблено серверну частину на NestJS з модульною доменно-орієнтованою архітектурою, безпечною автентифікацією на основі JWT із ротацією сесій, завантаженням і валідацією карт та пошуком найкоротшого шляху алгоритмом A* з підтримкою переходів між поверхами;
4) розроблено адаптивний двомовний клієнтський застосунок на Next.js і React зі схожим на Google Maps інтерфейсом, інтерактивною картою, пошуком, фільтруванням за категоріями, навігацією, історією, збереженими елементами, подіями та інтерфейсом адміністрування;
5) виконано модульне, інтеграційне та ручне тестування, яке підтвердило працездатність системи й відповідність сформульованим вимогам.

Розроблений застосунок має модульну архітектуру, відкриту до розширення: систему легко доповнити новими корпусами, категоріями приміщень та функціями без зміни наявного коду. Практична цінність роботи полягає у створенні зручного інструмента орієнтування, який знижує час пошуку приміщень для студентів, абітурієнтів і гостей університету.

Під час виконання роботи використано інструменти штучного інтелекту: системи Gemini Pro 3.1 Deep Research — для пошуку та інформації, модель Claude Opus 4.7 — для формування якісних і функціональних вимог до системи.

Вихідний код застосунку розміщено у загальнодоступному репозиторії системи контролю версій Git. Серверна та клієнтська частини організовані як монорепозиторій з робочими просторами npm.

Репозиторій проєкту https://github.com/juis6world/unimap.git

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Banks A., Porcello E. Learning GraphQL: Declarative Data Fetching for Modern Web Apps. Sebastopol : O`Reilly Media, 2018. 240 p.
2. Banks A., Porcello E. Learning React: Modern Patterns for Developing React Apps. 2nd ed. Sebastopol : O`Reilly Media, 2020. 350 p.
3. Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 4th ed. Cambridge : The MIT Press, 2022. 1312 p.
4. Evans E. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston : Addison-Wesley, 2003. 560 p.
5. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures : PhD dissertation. Irvine : University of California, 2000. 162 p.
6. Hart P. E., Nilsson N. J., Raphael B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics. 1968. Vol. 4, No. 2. P. 100–107.
7. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol : O`Reilly Media, 2017. 616 p.
8. Krug S. Don`t Make Me Think, Revisited: A Common Sense Approach to Web Usability. 3rd ed. Berkeley : New Riders, 2014. 216 p.
9. Newman S. Building Microservices: Designing Fine-Grained Systems. 2nd ed. Sebastopol : O`Reilly Media, 2021. 616 p.
10. Nielsen J. Usability Engineering. San Francisco : Morgan Kaufmann Publishers, 1993. 362 p.
11. Provos N., Mazières D. A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference. Monterey : USENIX Association, 1999. P. 81–91.
12. Russell S., Norvig P. Artificial Intelligence: A Modern Approach. 4th ed. Hoboken : Pearson, 2021. 1136 p.
13. BullMQ Documentation : official site. URL: https://docs.bullmq.io/.

14. Drizzle ORM Documentation : official site. URL: https://orm.drizzle.team/docs.
15. Fastify Documentation : official site. URL: https://fastify.dev/docs/latest/.
16. GraphQL Specification. URL: https://spec.graphql.org/.
17. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT) : RFC 7519. Internet Engineering Task Force, 2015. URL: https://datatracker.ietf.org/doc/html/rfc7519.
18. Material Design 3 : design system guidelines. Google. URL: https://m3.material.io/.
19. NestJS Documentation : official site. URL: https://docs.nestjs.com/.
20. Next.js Documentation : official site. Vercel. URL: https://nextjs.org/docs.
21. Node.js Documentation : official site. OpenJS Foundation. URL: https://nodejs.org/en/docs.
22. PostgreSQL 16 Documentation. The PostgreSQL Global Development Group. URL: https://www.postgresql.org/docs/.
23. React Documentation : official site. Meta Open Source. URL: https://react.dev/.
24. TanStack Query Documentation : official site. URL: https://tanstack.com/query/latest.
25. Redis Documentation : official site. URL: https://redis.io/docs/latest/.
26. Scalable Vector Graphics (SVG) 2 : W3C Candidate Recommendation. World Wide Web Consortium, 2018. URL: https://www.w3.org/TR/SVG2/.
27. Tailwind CSS Documentation : official site. URL: https://tailwindcss.com/docs.
28. TypeScript Documentation : official site. Microsoft. URL: https://www.typescriptlang.org/docs/.
29. Zod Documentation : official site. URL: https://zod.dev/.
30. Zustand Documentation. Poimandres. URL: https://docs.pmnd.rs/zustand/.